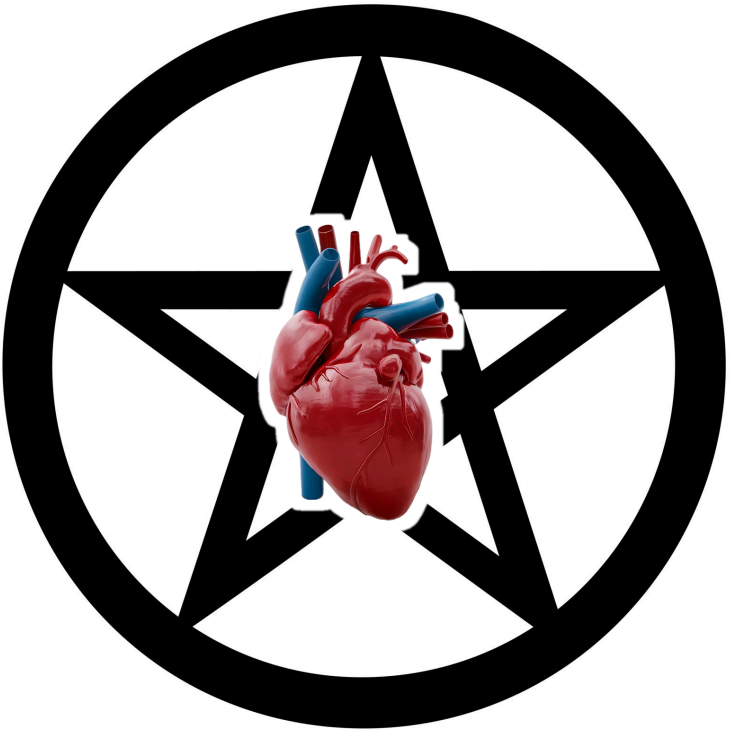
ZAGREVS

από τον Αλέξανδρο Γιαννάκη



Πέντε τεχνητές νοημοσύνες
Ένας βοηθός μέσα στον browser

Ακοή · Όραση · Λόγος · Νόημα · Πράξη

Ζαγρεύς · Οκτώβριος 2026

Πέντε μοντέλα, μία φωνή

Ο Ζαγρεύς είναι ένας βοηθός που ζει ολόκληρος μέσα σε μια σελίδα του browser. Ακούει ομιλία, βλέπει εικόνες, καταλαβαίνει τι του ζητάς, απαντά με κείμενο και μιλά. Δεν στέλνει τίποτα σε κάποιον server για υπολογισμό: ο server απλώς παραδίδει αρχεία, και όλη η σκέψη γίνεται στη συσκευή.

Κάθε ικανότητα προέρχεται από ένα διαφορετικό μοντέλο. Τα πέντε μοντέλα τρέχουν σε καθαρή JavaScript, το καθένα από δικό του repository, και ελέγχονται απέναντι σε υλοποιήσεις αναφοράς. Ο Ζαγρεύς τα ενώνει με έναν κοινό «εγκέφαλο», μια φωνή από το meSpeak.js και μια λειτουργία κλήσης που επιταχύνεται με WebAssembly.

Τι περιλαμβάνει αυτό το βιβλίο

Πέντε τεύχη, ένα για κάθε μοντέλο. Κάθε τεύχος εξηγεί τι κάνει το μοντέλο, πώς είναι φτιαγμένο, πώς εκτελείται στον browser και πώς το χρησιμοποιεί ο Ζαγρεύς. Τα αποσπάσματα κώδικα που σημειώνονται ως «διδακτικά» εξηγούν έναν μηχανισμό. Δεν είναι αντίγραφα του πραγματικού κώδικα.

Οι αριθμοί μετρήθηκαν σε υπολογιστή ανάπτυξης τον Οκτώβριο του 2026. Σε άλλη συσκευή οι χρόνοι αλλάζουν, αλλά τα αποτελέσματα των μοντέλων μένουν ίδια.

Το όνομα

Ο Ζαγρεύς της ορφικής παράδοσης διαμελίστηκε και ξαναγεννήθηκε από την καρδιά του, που σώθηκε. Το εξώφυλλο κρατά αυτή την εικόνα: πέντε κορυφές, μία για κάθε μοντέλο, και στο κέντρο η καρδιά που τα κάνει ένα πρόσωπο.

Χάρτης του βιβλίου

Τεύχος · σελίδες	Μοντέλο και θέμα
I · 1–16	**Ακοή · Cactus Whistle**
4–5	Η αρχιτεκτονική του Ζαγρέα
6–14	Από το μικρόφωνο στο κείμενο
15–16	WebAssembly και πρακτική δοκιμή
II · 17–32	**Όραση · Orasis-2**
17–24	Απόσταξη, δίκτυο, εικόνα
25–32	Runtimes, ετικέτες, μετρήσεις
III · 33–48	**Λόγος · SmolLM2-135M-Instruct**
33–43	Tokens, attention, παραγωγή
44–48	Νήματα, WebAssembly, συνομιλία
IV · 49–64	**Νόημα · multilingual MiniLM**
49–57	Embeddings προτάσεων
58–64	Προθέσεις, σημειώσεις, Wikipedia
V · 65–80	**Πράξη · Needle 2**
65–73	Κλήσεις εργαλείων
74–79	Κλήση, φωνή, πυρήνες, γλωσσάρι, πηγές

Πώς να το διαβάσεις

Τα τεύχη στέκονται και μόνα τους. Αν θέλεις τη συνολική εικόνα, ξεκίνα από τις σελίδες 4–5 και μετά διάλεξε μοντέλο. Το γλωσσάρι στη σελίδα 78 εξηγεί τους όρους που επαναλαμβάνονται.

Η αρχιτεκτονική του Ζαγρέα

Η σελίδα χωρίζεται σε τρία στρώματα. Η διεπαφή δείχνει τη συζήτηση και δέχεται είσοδο. Ο εγκέφαλος (`brain.js`) αποφασίζει τι ζητήθηκε και ποιος θα απαντήσει. Τα μοντέλα τρέχουν σε Web Workers, μακριά από το κύριο νήμα, ώστε η σελίδα να μην παγώνει όσο υπολογίζουν.

Είσοδος	Μοντέλο	Αποτέλεσμα
Ομιλία	Cactus Whistle	Κείμενο
Εικόνα	Orasis-2	Ετικέτες με scores
Κείμενο	MiniLM	Πρόθεση, ομοιότητα
Εντολή	Needle 2	Κλήση εργαλείου σε JSON
Ερώτηση	SmolLM2	Απάντηση σε κείμενο
Απάντηση	meSpeak.js	Ήχος WAV

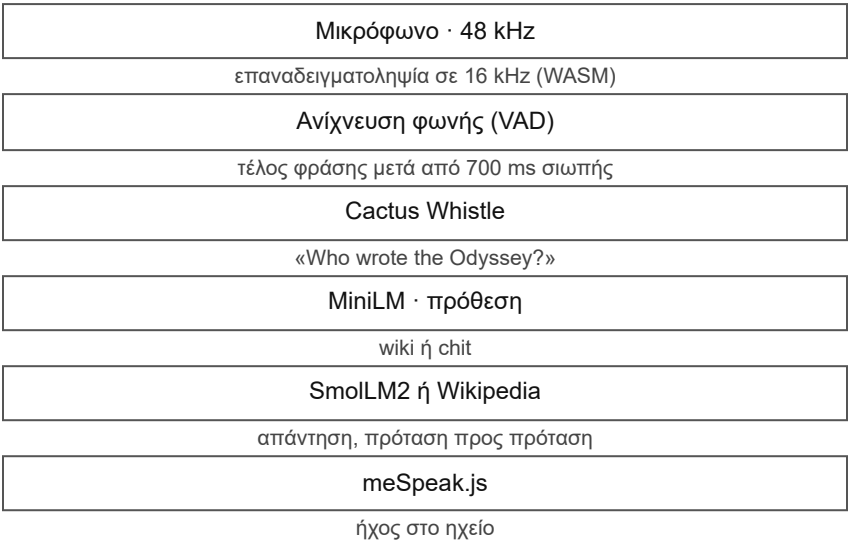
Ένας εγκέφαλος, πολλοί δρόμοι

Τα περισσότερα αιτήματα δεν χρειάζονται γλωσσικό μοντέλο. Η ώρα, η ημερομηνία, οι υπολογισμοί, οι υπενθυμίσεις και ο καιρός απαντώνται από κώδικα. Ο εγκέφαλος ρωτά πρώτα το MiniLM ποια είναι η πρόθεση. Μετά επιλέγει τη φθηνότερη διαδρομή που φτάνει.

```
// Διδακτικό σχήμα της απόφασης.
const intent = await classify(text);     // MiniLM
if (intent.startsWith('cmd_'))
  return command(intent, text);          // Needle 2
if (intent === 'wiki') return wikipedia(text);
if (intent === 'chit') return chat(text); // SmolLM2
return localReply(intent, text);         // κανόνες
```

Ένα αίτημα από άκρη σε άκρη

Η καλύτερη εισαγωγή στα πέντε μοντέλα είναι να ακολουθήσεις μια φράση από τη στιγμή που ειπώθηκε μέχρι τη στιγμή που ακούγεται η απάντηση. Ας πούμε ότι στη διάρκεια μιας κλήσης λες «Who wrote the Odyssey?».



Η ροή δεν περιμένει να τελειώσει όλη η απάντηση. Μόλις το γλωσσικό μοντέλο κλείσει την πρώτη πρόταση, το meSpeak αρχίζει να τη λέει, ενώ η επόμενη ακόμη γράφεται. Αν μιλήσεις πάνω από τον Ζαγρέα, η ομιλία του σταματά. Αυτό λέγεται barge-in.

Στο τεύχος V, σελίδα 74, η ίδια διαδρομή περιγράφεται με τους χρόνους της.

Cactus Whistle · η ακοή

Το Whistle μετατρέπει ομιλία σε κείμενο. Είναι μοντέλο της Cactus Compute, μόλις 16,9 MB, και διανέμεται ως αρχείο `whistle.cact`. Το repository *Cactus_Whistle-PureJS* το εκτελεί σε καθαρή JavaScript, χωρίς native κώδικα και χωρίς server, και δίνει τα ίδια κείμενα με την επίσημη μηχανή στα δείγματα ελέγχου.

Ιδιότητα	Τιμή
Μέγεθος	16,9 MB (.cact, κβαντισμένο)
Είσοδος	PCM 16 kHz, mono, τιμές στο [−1, 1]
Γλώσσες	en, de, fr, es, it, nl, pl, ή αυτόματη αναγνώριση
Λεξιλόγιο	8192 κομμάτια κειμένου
Encoder	8 blocks, πλάτος 512
Decoder	8 blocks, engram στα layers 3 και 7
Αναζήτηση	5 beams (προεπιλογή) ή greedy

Τρία στάδια

Το frontend μετατρέπει τον ήχο σε φάσμα mel. Ο encoder διαβάζει όλο το απόσπασμα και φτιάχνει μια αναπαράσταση ανά 80 ms. Ο decoder γράφει το κείμενο token προς token, κοιτώντας τον encoder σε κάθε βήμα.

Τα ελληνικά δεν είναι στις γλώσσες του Whistle. Γι' αυτό η υπαγόρευση και οι κλήσεις του Ζαγρέα δουλεύουν προς το παρόν στα αγγλικά, ενώ η γραπτή συζήτηση δέχεται και ελληνικά.

Από το μικρόφωνο στα 16 kHz

Τα μικρόφωνα των υπολογιστών και των κινητών δίνουν συνήθως 44,1 ή 48 kHz. Το Whistle θέλει 16 kHz, δηλαδή 16.000 δείγματα το δευτερόλεπτο. Η μετατροπή λέγεται επαναδειγματοληψία (resampling).

Ο Ζαγρεύς παίρνει τον ήχο με `getUserMedia` και έναν `ScriptProcessor` 2048 δειγμάτων. Κάθε κομμάτι περνά από έναν polyphase resampler γραμμένο σε Rust και μεταγλωττισμένο σε WebAssembly. Το αποτέλεσμα κόβεται σε πλαίσια των 20 ms, δηλαδή 320 δειγμάτων.

```
// Διδακτικό: windowed-sinc resampler.
// 16 taps σε κάθε πλευρά, παράθυρο Blackman,
// 256 φάσεις υπολογισμένες εκ των προτέρων.
for (let n = 0; n < outLen; n++) {
  const t = n * ratio;            // θέση στο input
  const i = Math.floor(t);
  const ph = Math.round((t - i) * 256);
  let acc = 0;
  for (let k = -15; k <= 16; k++)
    acc += x[i + k] * table[ph][k + 15];
  out[n] = acc;
}
```

Γιατί όχι γραμμική παρεμβολή

Η γραμμική παρεμβολή είναι γρήγορη, αλλά αφήνει συχνότητες πάνω από τα 8 kHz να «διπλώσουν» προς τα κάτω ως θόρυβος. Το φίλτρο sinc τις κόβει πριν μειωθεί ο ρυθμός. Ο πίνακας φάσεων έριξε τον χρόνο για 13 s ήχου από 460 ms σε 26 ms, με σφάλμα περίπου $3 \cdot 10^{-5}$.

Το frontend: από κύμα σε φάσμα

Το μοντέλο δεν ακούει το κύμα όπως είναι. Βλέπει πόση ενέργεια υπάρχει σε κάθε ζώνη συχνοτήτων, σε μικρά διαδοχικά παράθυρα. Αυτή η εικόνα λέγεται φασματογράφημα mel.

1. Κανονικοποίηση έντασης: το 99ο εκατοστημόριο της έντασης των block των 20 ms γίνεται 0,1.
2. Παράθυρα των 25 ms με βήμα 10 ms και συμμετρικό παράθυρο Hann.
3. FFT 512 σημείων και φάσμα ισχύος.
4. 80 φίλτρα mel από το ίδιο το αρχείο του μοντέλου.
5. `log(mel + 2⁻²⁴)` και κανονικοποίηση μέσης τιμής και διασποράς ανά κανάλι.

```
// Διδακτική περιγραφή του frontend.
for (const frame of frames(pcm, 400, 160)) {
  const w = hann(frame);           // 25 ms
  const p = powerSpectrum(fft512(w));
  const mel = melBank80(p);
  feats.push(mel.map(v => Math.log(v + 2**-24)));
}
normalisePerChannel(feats);
```

Η κλίμακα mel

Το ανθρώπινο αυτί ξεχωρίζει καλά τις χαμηλές συχνότητες και χειρότερα τις υψηλές. Τα φίλτρα mel είναι πυκνά στα χαμηλά και αραιά στα ψηλά, οπότε το μοντέλο ξοδεύει την προσοχή του εκεί που βρίσκεται η πληροφορία της ομιλίας.

Σιωπή, φωνή και τέλος φράσης

Υπάρχουν δύο διαφορετικοί έλεγχοι σιωπής. Ο πρώτος ανήκει στο Whistle. Αν η διαφορά ανάμεσα στο 99ο και στο 10ο εκατοστημόριο της έντασης, στη ζώνη 250–3500 Hz, είναι κάτω από 5 dB, το απόσπασμα θεωρείται σιωπή και δεν περνά από το μοντέλο.

Ο δεύτερος ανήκει στον Ζαγρέα και τρέχει σε κάθε πλαίσιο των 20 ms κατά τη διάρκεια μιας κλήσης: ένας ανιχνευτής φωνής (VAD) σε WebAssembly. Παρακολουθεί τη στάθμη του θορύβου και αποφασίζει πότε αρχίζει και πότε τελειώνει μια φράση.

Ρύθμιση	Τιμή	Ρόλος
Κατώφλι	10 dB	πόσο πάνω από τον θόρυβο είναι η φωνή
Τέλος φράσης	700 ms	σιωπή που κλείνει τη φράση
Preroll	15 πλαίσια	300 ms κρατιούνται πριν την έναρξη
Μέγιστο	700 πλαίσια	14 s ανά φράση
Barge-in	12 πλαίσια	240 ms φωνής διακόπτουν τον Ζαγρέα

Γιατί χρειάζεται το preroll

Ο ανιχνευτής καταλαβαίνει ότι άρχισες να μιλάς λίγο αφότου άρχισες. Χωρίς τα 300 ms που κρατά από πριν, η πρώτη συλλαβή θα χανόταν και το «Hello» θα ακουγόταν «llo». Και οι δύο ρυθμίσεις, κατώφλι και τέλος φράσης, αλλάζουν από τις ρυθμίσεις της εφαρμογής.

Το stem

Ανάμεσα στο φάσμα και στον encoder υπάρχει ένα μικρό στάδιο συνελίξεων, το stem. Μικραίνει την εικόνα του ήχου στον χρόνο και στη συχνότητα, ώστε ο encoder να δουλεύει σε λιγότερες και πλουσιότερες θέσεις.

Βήμα	Πράξη
1	Συνέλιξη 3×3, βήμα 2
2	Depthwise 3×3, βήμα 2, και κβαντισμένη pointwise
3	Depthwise 3×3, βήμα 2, και κβαντισμένη pointwise
4	SiLU μετά από κάθε στάδιο
5	Προβολή 10 × 128 → 512

Τρία βήματα των 2 μειώνουν τον χρόνο οκτώ φορές: από ένα πλαίσιο ανά 10 ms σε μία θέση ανά 80 ms. Οι 80 ζώνες mel γίνονται 10, η καθεμιά με 128 κανάλια, και η προβολή τις ενώνει σε ένα διάνυσμα 512 τιμών.

Depthwise και pointwise

Μια depthwise συνέλιξη φιλτράρει κάθε κανάλι χωριστά. Μια pointwise (1×1) αναμειγνύει τα κανάλια σε κάθε θέση. Μαζί κάνουν σχεδόν ό,τι μια πλήρης συνέλιξη, με πολύ λιγότερες πράξεις. Το ίδιο κόλπο εμφανίζεται ξανά στο Orasis-2, στο τεύχος ΙΙ.

```
// Διδακτικό: SiLU.
const silu = x => x / (1 + Math.exp(-x));
```

Ο encoder

Ο encoder διαβάζει ολόκληρο το απόσπασμα μονομιάς. Κάθε θέση μπορεί να κοιτάξει όλες τις άλλες, μπροστά και πίσω, γιατί το κείμενο δεν έχει γραφτεί ακόμη και δεν υπάρχει λόγος να κρυφτεί το μέλλον. Έχει 8 blocks, και καθένα περιέχει:

- μισό βήμα Hadamard MLP,
- attention χωρίς αιτιότητα, με GQA και RoPE,
- module συνέλιξης: pointwise προς 1024, GLU, depthwise 9 βημάτων στον χρόνο, κανονικοποίηση, SiLU, pointwise,
- δεύτερο μισό βήμα Hadamard MLP.

Η δομή «MLP – attention – συνέλιξη – MLP» θυμίζει τον Conformer. Το attention βλέπει μακρινές σχέσεις, η συνέλιξη τις κοντινές, όπως τη μετάβαση από φθόγγο σε φθόγγο.

Τέσσερις λωρίδες (mHC)

Αντί για ένα residual ρεύμα, ο encoder κρατά τέσσερα παράλληλα, που αναμειγνύονται με μάθηση ανάμεσα στα blocks. Η ίδια ιδέα υπάρχει και στο Needle 2.

Κόστος

Το attention μεγαλώνει με το τετράγωνο του μήκους. Ένα απόσπασμα 30 s κοστίζει περισσότερο από τρία των 10 s. Αποσπάσματα πάνω από 30 s κόβονται στο πιο ήσυχο σημείο των τελευταίων πέντε δευτερολέπτων κάθε παραθύρου.

Ο decoder

Ο decoder γράφει. Σε κάθε βήμα κοιτά τα tokens που έχει ήδη γράψει και την αναπαράσταση του encoder, και προτείνει πιθανότητες για το επόμενο κομμάτι κειμένου. Έχει 8 blocks:

- αιτιατό attention με GQA, με 3 βήματα συνέλιξης στα Q, K και V,
- cross attention με πύλη πάνω στο απόσπασμα: τα keys φέρουν ημιτονοειδές σήμα θέσης με πύλη,
- Hadamard MLP,
- αναζήτηση engram στα layers 3 και 7.

Engram: μνήμη από n-grams

Ένα engram είναι ένας πίνακας που διαβάζεται με βάση τα τελευταία tokens και όχι με πολλαπλασιασμούς. Τα πρόσφατα tokens γίνονται ένα hash, και το hash δείχνει μια γραμμή του πίνακα. Έτσι το μοντέλο θυμάται φτηνά συχνούς συνδυασμούς λέξεων.

Η κεφαλή εξόδου

Το Whistle μοιράζεται τον πίνακα embedding εισόδου με την κεφαλή εξόδου (tied embedding), κβαντισμένο στα 4 bit. Το ίδιο τραπέζι μετατρέπει ένα token σε διάνυσμα στην είσοδο και ένα διάνυσμα σε 8192 βαθμούς στην έξοδο.

```
// Διδακτικό βήμα του decoder.
let x = embed(lastToken);
for (const L of layers) x = L.step(x, encOut, cache);
const logits = tiedHead(x);   // 8192 τιμές
```

Κβάντιση: Cactus Quants

Τα 16,9 MB χωρούν ένα ολόκληρο μοντέλο ομιλίας επειδή τα βάρη δεν αποθηκεύονται ως αριθμοί των 32 bit. Πολλοί πίνακες έχουν μόλις 2 bit ανά βάρος. Κάθε τιμή δείχνει ένα κέντρο από ένα μικρό codebook, και τα κέντρα είναι ακέραιοι int8.

Περιστροφή Hadamard

Πριν από κάθε πολλαπλασιασμό, η ενεργοποίηση περιστρέφεται με πίνακα Hadamard σε ομάδες των 128 τιμών και κβαντίζεται σε int8 με μία κλίμακα ανά ομάδα. Η περιστροφή απλώνει τις ακραίες τιμές σε όλη την ομάδα, οπότε η κβάντιση χάνει λιγότερα. Τα Q, K, V και οι τιμές των engram κβαντίζονται σε int8 ανά κεφαλή.

Πίνακες αναζήτησης

Οι φαρδιοί πολλαπλασιασμοί δεν αποσυμπιέζουν ποτέ τα βάρη των 2 bit. Για κάθε ενεργοποίηση φτιάχνεται ένας πίνακας με όλα τα πιθανά αθροίσματα για ένα byte βαρών, και ο πολλαπλασιασμός γίνεται διαδοχικές αναγνώσεις.

```
// Διδακτικό: 4 βάρη των 2 bit σε ένα byte.
// lut[b] = άθροισμα των 4 γινομένων για byte b.
for (let b = 0; b < 256; b++)
  lut[b] = c[b&3]*a0 + c[b>>2&3]*a1
         + c[b>>4&3]*a2 + c[b>>6]*a3;
sum += lut[packed[j]];
```

Τα αποτελέσματα δεν είναι bit προς bit ίδια με την επίσημη μηχανή και δεν μπορούν να είναι. Μια διαφορά στο τελευταίο ψηφίο αλλάζει πού και πού μια στρογγυλοποίηση. Το cosine του encoder είναι 0,9998 ή καλύτερο, και τα κείμενα ταυτίζονται.

Αναζήτηση: greedy και beams

Μετά το token αρχής, το Whistle διαλέγει τη γλώσσα με argmax, εκτός αν του τη δώσεις. Ύστερα γράφει. Το ερώτημα είναι πόσες εκδοχές της πρότασης κρατά ζωντανές.

Τρόπος	Τι κάνει
Greedy (beams 1)	Κρατά πάντα το πιο πιθανό επόμενο token
Beam search (5)	Κρατά 5 υποψήφιες, με τις 6 καλύτερες συνέχειες καθεμιάς

Με 5 beams η αναζήτηση σταματά όταν τελειώσουν 5 υποθέσεις και κρατά εκείνη με το καλύτερο σκορ κανονικοποιημένο ως προς το μήκος. Μια υπόθεση που επαναλαμβάνει το τέλος της τέσσερις φορές κόβεται.

Η επιλογή του Ζαγρέα

Στις κλήσεις ο Ζαγρεύς χρησιμοποιεί greedy, γιατί η ταχύτητα μετρά περισσότερο. Το greedy κόβει τον χρόνο αποκωδικοποίησης περίπου τέσσερις φορές και τον συνολικό περίπου 40%. Η ρύθμιση «beams κλήσης» το αλλάζει.

```
// Διδακτικό σχήμα της κλήσης.
const r = await models.whistle.transcribe(pcm, {
  language: settings.sttLang,       // 'en'
  beams: settings.callBeams,        // 1
  onPartial: t => showLive(t)
});
```

Στη δοκιμή με το δείγμα των 11,7 s, η μεταγραφή μέσα σε κλήση πήρε 2,3 s. Ο λόγος πραγματικού χρόνου ήταν περίπου 0,2.

Το Whistle σε WebAssembly

Η καθαρή JavaScript του repository τρέχει παντού, ακόμη και σε παλιά κινητά. Ο Ζαγρεύς προσθέτει έναν πυρήνα σε WebAssembly για τη βαρύτερη πράξη: τον πολλαπλασιασμό κβαντισμένου πίνακα με διάνυσμα (`qmat_matvec`).

Ο πυρήνας είναι γραμμένος σε Rust για `wasm32-unknown-unknown` και χρησιμοποιεί εντολές SIMD των 128 bit. Τα ακέραια γινόμενα int8 αθροίζονται ακριβώς, και τα float αθροίζονται με την ίδια σειρά με τη JavaScript. Γι' αυτό τα αποτελέσματα είναι **bit προς bit ίδια**.

Μέτρηση	JS	WASM	Επιτάχυνση
beams 1	1,00	0,62	×1,61
beams 5	1,00	0,61	×1,65

Σχετικοί χρόνοι στο δείγμα των 11 s. Η έξοδος είναι ίδια και στις δύο περιπτώσεις.

Μόνο με SIMD

Χωρίς SIMD, ο πυρήνας WASM είναι πιο αργός από τους πίνακες αναζήτησης της JavaScript. Η σελίδα ελέγχει με `WebAssembly.validate` ένα μικρό module που περιέχει εντολή SIMD. Αν δεν περάσει, το μοντέλο μένει σε JavaScript.

```
// Πώς «κουμπώνει» ο πυρήνας (διδακτικό σχήμα).
QMat.prototype.apply = function (x, out) {
  wasm.qmat_matvec(this.ptr, xPtr, outPtr, ...);
};
```

Πρακτική δοκιμή · η ακοή

Μια μικρή διαδικασία

1. Τρέξε `npm start` και άνοιξε τη σελίδα.
2. Περίμενε να φορτώσουν τα βασικά μοντέλα στο πλαϊνό μενού.
3. Πάτησε το μικρόφωνο, πες μια σύντομη αγγλική φράση και πάτησέ το ξανά.
4. Άνοιξε τις ρυθμίσεις και τρέξε τη σύγκριση JS–WASM με το δείγμα.
5. Ξεκίνα μια κλήση και άφησε μια παύση ενός δευτερολέπτου στο τέλος.

Τι αξίζει να παρατηρήσεις

Το πρώτο τρέξιμο είναι πιο αργό, γιατί η μηχανή JavaScript δεν έχει ακόμη βελτιστοποιήσει τον κώδικα. Ο θόρυβος του χώρου αλλάζει το σημείο που η κλήση θεωρεί ότι τελείωσες. Αν σε κόβει νωρίς, αύξησε το «τέλος φράσης».

```
// Φύλλο παρατήρησης · συμπλήρωσέ το.
{
  "device": "",
  "browser": "",
  "phrase": "",
  "transcript": "",
  "sttMs": null,
  "accel": "wasm | js",
  "endMs": 700
}
```

Στο επόμενο τεύχος ο Ζαγρεύς ανοίγει τα μάτια του: μια εικόνα γίνεται 1.280 αριθμοί, και οι αριθμοί γίνονται λέξεις.

Orasis-2 · η όραση

Το Orasis-2 μετατρέπει μια εικόνα σε 1.280 αριθμούς, ένα embedding. Ένας μικρός γραμμικός χάρτης μεταφέρει αυτούς τους αριθμούς στον χώρο ενός μεγάλου μοντέλου CLIP. Εκεί η εικόνα συγκρίνεται με μια τράπεζα 1.253 λέξεων, και οι λέξεις που βρίσκονται πιο κοντά γίνονται οι ετικέτες της.

Ιδιότητα	Τιμή
Βάση	MobileNetV4 Small (timm)
Δάσκαλος	OpenCLIP ViT-B-16, laion2b
Παράμετροι	2,48 εκατομμύρια
Αρχείο encoder	5,0 MB (encoder.ors, 16 bit)
Είσοδος	224 × 224 RGB
Έξοδος	1.280 τιμές, μήκος 1
Χάρτης κειμένου	1.280 → 512 (textmap.orh)
Τράπεζα λέξεων	1.253 λέξεις, 512 διαστάσεις
Runtimes	JavaScript, WebAssembly, Rust

Το repository

Το *zagreus-orasis* φτιάχτηκε για τον Ζαγρέα. Περιέχει τα μοντέλα, τον κώδικα εκπαίδευσης στο Google Colab, τρία runtimes χωρίς εξαρτήσεις και όλες τις μετρήσεις, με τα καλά και τα άσχημα νέα τους.

Αυτό το τεύχος εξηγεί πώς ένα μικρό δίκτυο μαθαίνει από ένα μεγάλο, πώς εκτελείται στον browser και τι μπορεί και τι δεν μπορεί να δει.

Γιατί ένα νέο μοντέλο όρασης

Η πρώτη έκδοση του Ζαγρέα έβλεπε με το MobileCLIP-S0 της Apple. Ήταν ακριβές, αλλά είχε δύο προβλήματα.

- **Η άδεια.** Οι όροι της Apple για τα βάρη καλύπτουν μόνο εκπαιδευτική και ακαδημαϊκή χρήση.
- **Το μέγεθος.** Τα βάρη FP32 πιάνουν 45,5 MB, και μια εικόνα ήθελε περίπου 4 s σε υπολογιστή.

Το Orasis-2 είναι η απάντηση σε αυτά τα δύο. Είναι περίπου εννέα φορές μικρότερο και τρέχει σε περίπου 0,1 s με WebAssembly SIMD.

	MobileCLIP-S0	**Orasis-2**
Αρχείο	45,5 MB	5,0 MB
Λέξεις, χωρίς εκπαίδευση	62,5%	39,6%
Με εκπαιδευμένο κεφάλι	69,3%	54,3%
Χρόνος στον Ζαγρέα	~4 s	~0,1 s

Μέση ακρίβεια σε 14 κατηγορίες τοπίων και αντικειμένων, σε 280 εικόνες που δεν είχαν χρησιμοποιηθεί πριν. Τυχαία επιλογή: 7,1%.

> Το Orasis-2 **δεν** φτάνει ακόμη το MobileCLIP-S0. Είναι ένας συμβιβασμός: λιγότερη ακρίβεια με αντάλλαγμα μέγεθος, ταχύτητα και καθαρότερους όρους χρήσης.

Εικόνες και λέξεις στον ίδιο χώρο

Το CLIP (Contrastive Language–Image Pretraining) εκπαιδεύει δύο encoders μαζί: έναν για εικόνες και έναν για κείμενα. Τους δείχνει εκατομμύρια ζεύγη εικόνας και λεζάντας και τους ζητά ένα πράγμα: το διάνυσμα μιας εικόνας να είναι κοντά στο διάνυσμα της δικής της λεζάντας και μακριά από τις λεζάντες των άλλων εικόνων.

Στο τέλος, οι δύο encoders μιλούν την ίδια «γλώσσα» αριθμών. Η φωτογραφία ενός σκύλου και η φράση «a photo of a dog» καταλήγουν σε γειτονικά σημεία ενός κοινού χώρου.

Δανεικός χώρος

Το Orasis-2 δεν εκπαιδεύτηκε με λεζάντες. Μαθαίνει να βάζει κάθε εικόνα στο σημείο όπου θα την έβαζε ο encoder εικόνας ενός μεγάλου CLIP, του OpenCLIP ViT-B-16. Αν το πετύχει, κληρονομεί δωρεάν και τον encoder κειμένου του: οι λέξεις μπορούν να υπολογιστούν μία φορά με το μεγάλο μοντέλο και να αποθηκευτούν.

Ταξινόμηση χωρίς εκπαίδευση

Αυτό επιτρέπει zero-shot ταξινόμηση. Γράφεις τις κατηγορίες ως λέξεις, και κάθε εικόνα συγκρίνεται μαζί τους. Η τράπεζα του Ζαγρέα έχει 1.253 λέξεις, από «volcano» μέχρι «teddy bear».

> Το Orasis-2 δεν περιγράφει εικόνες με προτάσεις. Μόνο κατατάσσει λέξεις που του δίνεις. Αν μια λέξη δεν υπάρχει στην τράπεζα, δεν μπορεί να εμφανιστεί.

Απόσταξη: ο δάσκαλος και ο μαθητής

Η απόσταξη γνώσης (knowledge distillation) εκπαιδεύει ένα μικρό μοντέλο, τον μαθητή, να μιμείται ένα μεγάλο, τον δάσκαλο. Δεν χρειάζονται ετικέτες. Αρκούν εικόνες: ο δάσκαλος λέει πού πάει κάθε εικόνα, και ο μαθητής προσπαθεί να τη στείλει εκεί.

Ρόλος	Μοντέλο
Δάσκαλος	OpenCLIP ViT-B-16, laion2b_s34b_b88k (MIT)
Μαθητής	MobileNetV4 Small, προεκπαιδευμένο στο ImageNet
Εικόνες	241.690 από το COCO 2017 (train και unlabeled)
Εκπαίδευση	8 εποχές στο Google Colab

Όχι από το μηδέν

Τα προηγούμενα πειράματα, Orasis-S και Orasis-M, ξεκίνησαν από τυχαία βάρη. Το Orasis-2 ξεκινά από ένα δίκτυο που ήδη αναγνωρίζει τα 1.000 αντικείμενα του ImageNet. Αυτό έκανε τη μεγαλύτερη διαφορά.

Επαυξήσεις

Κάθε εικόνα περνά με τυχαία περικοπή (40–100% της επιφάνειας), τυχαίο καθρέφτισμα και αλλαγή φωτεινότητας και κορεσμού κατά ±25%. Ο δάσκαλος βλέπει την ίδια παραλλαγή με τον μαθητή. Έτσι ο μαθητής βλέπει την ίδια εικόνα διαφορετική σε κάθε εποχή.

Ήρεμη αρχή

Την πρώτη εποχή εκπαιδεύεται μόνο ο χάρτης προς τον χώρο του CLIP. Η ραχοκοκαλιά μένει σταθερή και αρχίζει να μαθαίνει σταδιακά στη δεύτερη εποχή. Έτσι τα καλά προεκπαιδευμένα βάρη δεν χαλάνε από τα πρώτα, χαοτικά βήματα.

Τέσσερις στόχοι σε μία συνάρτηση

Ο μαθητής δεν έχει έναν στόχο αλλά τέσσερις, και το κόστος τους αθροίζεται με βάρη.

```
// Συντομευμένο από το colab/orasis2.py.
cos  = 1 - mean(zs · zt)
soft = KL( softmax(100 zs·bankᵀ) ‖ softmax(100 zt·bankᵀ) )
nce  = crossEntropy(100 zs·ztᵀ, διαγώνιος)
keep = 4 · KL( imagenet(μαθητής)/2 ‖ imagenet(αρχικό)/2 )
loss = cos + 0.5·soft + 0.1·nce + 0.2·keep
```

Όρος	Τι ζητά
cos	το διάνυσμα του μαθητή να δείχνει όπου και του δασκάλου
soft	η κατάταξη των 1.253 λέξεων να είναι ίδια
nce	κάθε εικόνα να ξεχωρίζει από τις άλλες του batch
keep	να μην ξεχάσει τις κατηγορίες του ImageNet

Ο όρος `soft` είναι ο πιο ενδιαφέρων. Δεν αρκεί να είναι το διάνυσμα «σχεδόν» σωστό. Μετρά πώς αυτό το «σχεδόν» αλλάζει τη σειρά των λέξεων, γιατί αυτή τη σειρά βλέπει τελικά ο χρήστης.

Ο όρος `keep` κρατά ζωντανό το παλιό ταλέντο του δικτύου. Χωρίς αυτόν, η προσαρμογή στο CLIP θα έσβηνε σιγά-σιγά τη γνώση του ImageNet.

Η ραχοκοκαλιά: MobileNetV4 Small

Το MobileNetV4 σχεδιάστηκε για κινητά. Είναι αποκλειστικά συνελικτικό, χωρίς attention, και χρησιμοποιεί μόνο ReLU. Στη μορφή του Ζαγρέα είναι 58 πράξεις.

Πράξη	Πλήθος	Ρόλος
Pointwise 1×1	28	αναμειγνύει κανάλια
Depthwise	16	φιλτράρει κάθε κανάλι στον χώρο
Πλήρης 3×3	3	τα πρώτα στάδια, με βήμα 2
Πρόσθεση	10	συνδέσεις residual
Μέσος όρος	1	από χάρτη σε διάνυσμα

Το σχήμα του δικτύου

Η εικόνα 224 × 224 μικραίνει βήμα-βήμα στον χώρο, ενώ τα κανάλια πληθαίνουν: από 3 χρώματα σε 32, και στο τέλος σε 960. Ένας μέσος όρος σε όλο τον χάρτη (global average pooling) δίνει ένα διάνυσμα, και μια τελευταία pointwise το ανεβάζει στις 1.280 τιμές.

Depthwise και pointwise

Μια depthwise συνέλιξη φιλτράρει κάθε κανάλι χωριστά. Μια pointwise αναμειγνύει τα κανάλια σε κάθε θέση. Μαζί κάνουν σχεδόν ό,τι μια πλήρης συνέλιξη, με πολύ λιγότερες πράξεις. Το ίδιο κόλπο χρησιμοποιεί και το stem του Whistle.

Συνολικά το δίκτυο κάνει περίπου 185 εκατομμύρια πολλαπλασιασμούς ανά εικόνα.

Η μορφή .ors

Το ONNX περιγράφει ένα μοντέλο με εκατοντάδες είδη πράξεων. Το Orasis χρειάζεται μόνο τέσσερα. Γι' αυτό έχει τη δική του μικρή μορφή αρχείου: μια κεφαλίδα, έναν πίνακα πράξεων και τα βάρη.

```
// Η μορφή encoder.ors (little-endian).
"ORS1"                        // μαγική λέξη
u16 nops, nregs, size, dim    // 58, 59, 224, 1280
nops × 20 bytes:              // μία εγγραφή ανά πράξη
  kind, a, b, out, ci, co,
  k, stride, groups, act
βάρη: float 16 bit
```

kind	Πράξη
0	συνέλιξη με bias (κάθε μέγεθος, βήμα, ομάδες)
1	πρόσθεση δύο χαρτών
2	κλιμάκωση καναλιών
3	μέσος όρος όλου του χάρτη

Η ενεργοποίηση (καμία, ReLU, hard-sigmoid ή hard-swish) είναι πεδίο της πράξης, όχι ξεχωριστή πράξη.

Τι έγινε στην εξαγωγή

Τα BatchNorm ενώθηκαν με τις συνελίξεις, γιατί στην εκτέλεση είναι απλώς ένας πολλαπλασιασμός και μια πρόσθεση ανά κανάλι. Τα βάρη αποθηκεύτηκαν σε 16 bit. Το cosine ανάμεσα στην έξοδο των 16 και των 32 bit είναι 0,999994.

Προετοιμασία της εικόνας

Η εικόνα μπορεί να έρθει από την κάμερα, από αρχείο ή με σύρσιμο. Ο Ζαγρεύς τη μικραίνει πρώτα ώστε η μικρή της πλευρά να είναι το πολύ 512 pixel και στέλνει τα bytes RGB στον worker. Εκεί γίνονται τα βήματα του runtime:

1. αλλαγή μεγέθους ώστε η μικρή πλευρά να γίνει 256, με φίλτρο που αποφεύγει το aliasing,
2. κεντρική περικοπή 224 × 224,
3. κανονικοποίηση με τη μέση τιμή και την τυπική απόκλιση του CLIP.

```
// Από το orasis.js: κανονικοποίηση και NCHW.
for (c = 0; c < 3; c++)
  for (i = 0, j = c; i < n; i++, j += 3)
    input[c * n + i] = (rgb[j] / 255 - MEAN[c]) / STD[c];
```

Δύο κανονικοποιήσεις

Ο δάσκαλος περιμένει την κανονικοποίηση του CLIP. Το MobileNetV4 εκπαιδεύτηκε με του ImageNet. Αντί να υπάρχουν δύο εκδοχές του κώδικα, η εξαγωγή πρόσθεσε στην αρχή του δικτύου μια depthwise συνέλιξη 1×1. Αυτή μετατρέπει τη μία κανονικοποίηση στην άλλη. Είναι η πρώτη από τις 58 πράξεις.

Η σειρά NCHW

Ο canvas δίνει τα pixel ως RGBA, το ένα μετά το άλλο. Το δίκτυο θέλει πρώτα όλο το κόκκινο κανάλι, μετά όλο το πράσινο και μετά όλο το μπλε. Συνολικά 150.528 αριθμοί.

Ο executor σε JavaScript

Ο runtime σε JavaScript είναι περίπου 17 KB κώδικα ES5 με typed arrays. Δουλεύει σε παλιούς browsers, σε Web Workers και στο Node.

Καταχωρητές

Κάθε πράξη διαβάζει έναν ή δύο «καταχωρητές» και γράφει σε έναν νέο. Ο καταχωρητής 0 είναι η εικόνα και ο τελευταίος το διάνυσμα. Κατά τη φόρτωση, ο runtime ελέγχει όλα τα σχήματα και σημειώνει πότε χρησιμοποιείται τελευταία φορά κάθε καταχωρητής.

```
// Συντομευμένο από το orasis.js.
for (i = 0; i < ops.length; i++) {
  var dst = this.take(size(op.o));   // από το pool
  if (op.kind === 0) conv(op, W, regs[op.a], dst);
  else if (op.kind === 1) add(regs[op.a], regs[op.b], dst);
  ...
  if (op.act === 1) relu(dst);
  regs[op.o] = dst;
  if (lastUse[op.a] === i) this.give(regs[op.a]);
}
```

Ανακύκλωση μνήμης

Μόλις ένας χάρτης δεν χρειάζεται πια, ο πίνακάς του επιστρέφει σε μια δεξαμενή, οργανωμένη ανά μέγεθος. Η επόμενη πράξη που θέλει ίδιο μέγεθος τον ξαναχρησιμοποιεί. Έτσι ο garbage collector σχεδόν δεν δουλεύει, και η μνήμη μένει σταθερή από εικόνα σε εικόνα.

Η συνέλιξη

Οι συνελίξεις 1×1 με βήμα 1 έχουν δικό τους σύντομο βρόχο. Οι υπόλοιπες υπολογίζουν από πριν ποιο τμήμα κάθε γραμμής πέφτει μέσα στην εικόνα, ώστε ο εσωτερικός βρόχος να μην ελέγχει όρια.

Rust και WebAssembly SIMD

Ο ίδιος υπολογισμός υπάρχει και σε Rust χωρίς εξαρτήσεις. Μεταγλωττίζεται σε δύο αρχεία WebAssembly: `orasis.wasm` (34 KB) και `orasis-simd.wasm` (38 KB), με εντολές SIMD των 128 bit.

Runtime	Node, 1 νήμα	Ζαγρεύς, browser
JavaScript	317 ms	433 ms
WebAssembly	149 ms	–
WebAssembly SIMD	94 ms	~110 ms

Μετρήσεις σε υπολογιστή ανάπτυξης, Νοέμβριος 2026. Ο χρόνος του browser περιλαμβάνει την προετοιμασία της εικόνας και τη σύγκριση με τις λέξεις.

Ίδια αποτελέσματα

Και τα τρία runtimes αναπαράγουν το διάνυσμα αναφοράς της εξαγωγής από την Python με cosine τουλάχιστον 0,9999998. Δεν είναι bit προς bit ίδια, γιατί αθροίζουν float με διαφορετική σειρά. Στις 1.253 λέξεις όμως δίνουν τις ίδιες ετικέτες.

Η επιλογή

Ο worker του Ζαγρέα ελέγχει πρώτα αν ο browser υποστηρίζει SIMD. Αν ναι, φορτώνει την έκδοση SIMD, αλλιώς την απλή. Αν το WebAssembly λείπει ή έχει κλείσει από τις ρυθμίσεις, χρησιμοποιεί τη JavaScript. Το μοντέλο και οι λέξεις είναι τα ίδια σε όλες τις περιπτώσεις.

`node tools/test_orasis.js` ελέγχει και τα τρία απέναντι στο διάνυσμα αναφοράς.

Από το embedding στην ετικέτα

Η έξοδος του δικτύου είναι 1.280 αριθμοί με μήκος 1. Για να γίνουν λέξεις, ακολουθούνται τέσσερα βήματα:

1. ο χάρτης κειμένου (1.280 → 512, με bias) μεταφέρει το διάνυσμα στον χώρο του CLIP,
2. το νέο διάνυσμα κανονικοποιείται σε μήκος 1,
3. υπολογίζεται εσωτερικό γινόμενο με καθεμιά από τις 1.253 λέξεις,
4. οι λέξεις ταξινομούνται και κρατιούνται οι πρώτες.

```
// Από τον worker του Ζαγρέα.
var s = OrasisHead.scores(textMap, z);   // 512
var v = normalize(s);
return Orasis.topK(v, bank, k);          // cosine
```

Η τράπεζα

Τα 1.253 διανύσματα λέξεων υπολογίστηκαν μία φορά με τον encoder κειμένου του δασκάλου. Αποθηκεύονται σε 16 bit στο `bank.f16` (1,3 MB), και τα ονόματα στο `labels.json`.

Ομοιότητα, όχι πιθανότητα

Τα scores είναι cosine, από −1 έως 1. Στην πράξη είναι μικρά, συνήθως 0,15–0,30, και πολύ κοντά μεταξύ τους. Ένα score 0,24 δεν σημαίνει «24% βέβαιο». Σημαίνει μόνο ότι αυτή η λέξη ήταν πιο κοντά στην εικόνα από τις άλλες.

Πώς δείχνει ο Ζαγρεύς τα scores

Τα ακατέργαστα cosine δεν διαβάζονται εύκολα: 0,241, 0,232, 0,227. Ο Ζαγρεύς τα μετατρέπει σε σχετικά ποσοστά με softmax και κλίμακα 100. Είναι η ίδια κλίμακα που χρησιμοποίησε ο όρος `soft` στην εκπαίδευση.

```
// Από cosine σε σχετικό ποσοστό.
const top = res.top.slice(0, 5);
const best = top[0].score;
const ex = top.map(x => Math.exp(100 * (x.score - best)));
const sum = ex.reduce((a, b) => a + b, 0);
const dist = top.map((x, i) =>
  ({ label: x.label, p: ex[i] / sum }));
```

Η κλίμακα 100 μεγαλώνει τις μικρές διαφορές. Διαφορά 0,01 στο cosine γίνεται περίπου 2,7 φορές μεγαλύτερο βάρος.

Ένα πραγματικό παράδειγμα

Μια αεροφωτογραφία παραλίας από τα δείγματα του repository, όπως την είδε ο Ζαγρεύς:

Λέξη	cosine	σχετικό
coast	0,241	48%
beach	0,232	20%
seashore	0,227	12%
ocean	–	10%
sea	–	9%

Το ποσοστό είναι σχετικό με τις πέντε πρώτες λέξεις, όχι πραγματική βεβαιότητα. Αν η σωστή λέξη λείπει από την τράπεζα, κάποια άλλη θα πάρει υψηλό ποσοστό.

Ειλικρινείς μετρήσεις

Το repository μετρά τα μοντέλα σε εικόνες από τα Wikimedia Commons, σε 14 κατηγορίες: παραλία, βάρκα, σπηλιά, γκρεμός, έρημος, λουλούδι, δάσος, παγετώνας, λίμνη, βουνό, ποτάμι, ηφαίστειο, καταρράκτης, πυρκαγιά.

Μέθοδος (νεότερο τεστ, 280 εικόνες)	**Ακρίβεια**
Δάσκαλος ViT-B-16 + LDA	71,1%
MobileCLIP-S0 + LDA	69,3%
MobileCLIP-S0, λέξεις	62,5%
MobileNetV4 + LDA	56,4%
Orasis-2 + LDA	**54,3%**
Orasis-2, λέξεις	**39,6%**

Πόσο να τα εμπιστευτείς

Υπάρχουν τρία σύνολα ελέγχου. Το νεότερο ανοίχτηκε μία μόνο φορά, με όλες τις ρυθμίσεις αποφασισμένες από πριν. Είναι το πιο καθαρό. Τα παλιότερα είχαν ανοιχτεί πολλές φορές και είχαν επηρεάσει επιλογές, οπότε είναι πιο αισιόδοξα. Διαφορές λίγων μονάδων σε 280 εικόνες είναι μέσα στον θόρυβο.

Το ηφαίστειο

Με λέξεις χωρίς εκπαίδευση, το Orasis-2 δεν είπε «volcano» σε καμία από τις 20 εικόνες ηφαιστείων. Σε μια έκρηξη βλέπει «fire» και σε ένα ηφαίστειο στη δύση «sunset». Στο Imagenette, δέκα καθημερινά αντικείμενα, φτάνει όμως το 94% με λέξεις.

> Οι αποτυχίες δημοσιεύονται μαζί με τις επιτυχίες. Ένας αριθμός χωρίς τις συνθήκες του λέει λιγότερα απ' όσα φαίνεται.

Ένα κεφάλι που μαθαίνει στη συσκευή

Οι λέξεις δουλεύουν χωρίς εκπαίδευση, αλλά δεν είναι ο μόνος τρόπος. Αν έχεις λίγα παραδείγματα από τις δικές σου κατηγορίες, μπορείς να εκπαιδεύσεις ένα γραμμικό κεφάλι πάνω στα 1.280 χαρακτηριστικά, μέσα στον browser, σε κλάσματα του δευτερολέπτου.

Shrinkage LDA

Η γραμμική διακριτική ανάλυση (LDA) υπολογίζει τον μέσο όρο κάθε κατηγορίας και τη διασπορά γύρω από αυτόν. Με λίγα παραδείγματα, ο πίνακας διασποράς είναι αναξιόπιστος. Γι' αυτό «συρρικνώνεται» προς έναν απλό διαγώνιο πίνακα, εδώ κατά 0,6.

```
// Από το head.js του repository.
const head = OrasisHead.fitLDA(x, y, classes, 1280, 0.6);
const best = OrasisHead.top(head, z, 1);
const file = OrasisHead.save(head);   // .orh, 16 bit
```

Η εκπαίδευση είναι μία γραμμική επίλυση με παραγοντοποίηση Cholesky. Όταν τα παραδείγματα είναι λιγότερα από τις διαστάσεις, ο κώδικας λύνει ένα μικρότερο σύστημα με την ταυτότητα Woodbury.

Τέσσερις όψεις ανά εικόνα

Κάθε εικόνα εκπαίδευσης δίνει τέσσερα διανύσματα: το κέντρο, το κέντρο καθρεφτισμένο και τις δύο άκρες της μεγάλης πλευράς. Με περίπου 130 εικόνες ανά κατηγορία, το Orasis-2 φτάνει το 54,3%.

Ο Ζαγρεύς σήμερα χρησιμοποιεί μόνο τις λέξεις. Ένα κεφάλι για δικές σου κατηγορίες είναι φυσικό επόμενο βήμα.

Τα όρια της όρασης

1. Κλειστό λεξιλόγιο

Το μοντέλο διαλέγει μόνο από τις 1.253 λέξεις της τράπεζας. Ένα αντικείμενο που δεν υπάρχει εκεί θα πάρει το όνομα της πιο κοντινής λέξης, ακόμη κι αν είναι λάθος.

2. Τοπία

Το COCO έχει κυρίως ανθρώπους, ζώα και αντικείμενα της καθημερινότητας, και λίγα τοπία. Στις εικόνες του COCO ο μαθητής ακολουθεί τον δάσκαλο με cosine 0,733. Στα τοπία των Commons μόνο με 0,595. Ευρύτερα δεδομένα εκπαίδευσης είναι το επόμενο πείραμα, αλλά δεν έχει δοκιμαστεί.

3. Η περικοπή

Η κεντρική περικοπή και η μικρή ανάλυση (224 × 224) σβήνουν λεπτομέρειες. Κείμενο, μικρά αντικείμενα και ό,τι βρίσκεται στις άκρες συχνά χάνονται.

4. Όροι χρήσης

Ο κώδικας είναι MIT. Τα βάρη ξεκινούν από ένα μοντέλο Apache 2.0 που εκπαιδεύτηκε στο ImageNet-1k, του οποίου οι όροι περιορίζουν τα δεδομένα σε μη εμπορική έρευνα. Η άδεια των βαρών δεν λύνει αυτό το ερώτημα, οπότε για εμπορική χρήση χρειάζεται νομική συμβουλή.

Και ένα θετικό

Όλα γίνονται στη συσκευή. Η εικόνα της κάμερας δεν ανεβαίνει πουθενά και υπάρχει μόνο στη μνήμη της σελίδας. Η κάμερα χρειάζεται `localhost` ή HTTPS (σελίδα 77).

Πρακτική δοκιμή · η όραση

Μια μικρή διαδικασία

1. Φόρτωσε το Orasis-2 από το πλαϊνό μενού και σημείωσε τι γράφει δίπλα: `wasm-simd`, `wasm` ή `js`.
2. Δοκίμασε μια εικόνα με ένα καθαρό αντικείμενο στο κέντρο, για παράδειγμα μια κούπα ή έναν σκύλο.
3. Δοκίμασε μια φωτογραφία τοπίου, όπως ένα βουνό ή μια λίμνη.
4. Δοκίμασε την ίδια εικόνα περικομμένη ώστε το θέμα να είναι στην άκρη.
5. Κλείσε το WebAssembly στις ρυθμίσεις, ξαναφόρτωσε και σύγκρινε τον χρόνο.

Τι αξίζει να παρατηρήσεις

Τα αντικείμενα αναγνωρίζονται καλύτερα από τα τοπία, όπως προβλέπουν οι μετρήσεις. Χωρίς WebAssembly οι λέξεις μένουν ίδιες, και αλλάζει μόνο ο χρόνος.

```
// Φύλλο παρατήρησης · συμπλήρωσέ το.
{
  "kernel": "",
  "objectImage": { "top": [], "ms": null },
  "landscapeImage": { "top": [], "ms": null },
  "croppedResult": "",
  "jsMs": null
}
```

Στο επόμενο τεύχος ο Ζαγρεύς μιλά: ένα γλωσσικό μοντέλο 135 εκατομμυρίων παραμέτρων γράφει απαντήσεις, token προς token, ίδιες με του ONNX Runtime.

SmolLM2-135M-Instruct · ο λόγος

Το SmolLM2-135M-Instruct είναι ένα μικρό γλωσσικό μοντέλο του Hugging Face, εκπαιδευμένο να ακολουθεί οδηγίες και να συνομιλεί. Με 135 εκατομμύρια παραμέτρους είναι περίπου χίλιες φορές μικρότερο από τα μεγάλα μοντέλα συνομιλίας. Χωράει σε ένα κινητό και απαντά σε απλές ερωτήσεις στα αγγλικά.

Ιδιότητα	Τιμή
Αρχείο	onnx-community/SmolLM2-135M-Instruct-ONNX-MHA, model_quantized.onnx
Κβάντιση	q8 (uint8)
Βάρη	136,7 MB σε 11 αρχεία
Συνολική λήψη	~139 MB με τον tokenizer
Παραγωγή	greedy, repetition penalty 1,12

Το repository

Το *SmolLM2_135MInstruct-PureJS* εκτελεί το μοντέλο σε JavaScript ES5 με typed arrays, ώστε να δουλεύει ακόμη και σε παλιούς browsers. Δεν είναι προσέγγιση: δίνει **τις ίδιες απαντήσεις, token προς token,** με το Transformers.js και το ONNX Runtime Web σε WebAssembly. Τα logits βγαίνουν bit προς bit ίδια.

Αυτό το τεύχος εξηγεί πώς ένα κείμενο γίνεται tokens, τι συμβαίνει μέσα στα 30 layers, πώς επιλέγεται κάθε επόμενη λέξη και πώς ο Ζαγρεύς χρησιμοποιεί το μοντέλο σε συζήτηση και σε κλήση.

Ένα μοντέλο που μαντεύει τη συνέχεια

Ένα γλωσσικό μοντέλο κάνει ένα μόνο πράγμα: δεδομένου ενός κειμένου, δίνει μια βαθμολογία σε κάθε πιθανό επόμενο token. Για να γράψει μια απάντηση, το κάνει ξανά και ξανά. Διαλέγει ένα token, το προσθέτει στο κείμενο και ρωτά πάλι.

```
// Διδακτική παραγωγή.
let ids = tokenize(prompt);
for (let step = 0; step < maxNew; step++) {
  const logits = model(ids);      // 49.152 τιμές
  const next = argmax(logits);
  if (next === IM_END) break;
  ids.push(next);
}
```

Decoder-only

Σε αντίθεση με το Whistle, εδώ δεν υπάρχει χωριστός encoder. Το prompt και η απάντηση είναι μία ακολουθία. Κάθε θέση βλέπει μόνο τις προηγούμενες. Αυτό λέγεται αιτιατή προσοχή (causal attention).

Prefill και decode

Η εκτέλεση έχει δύο φάσεις. Στο prefill, όλο το prompt περνά μαζί από το μοντέλο. Αυτό είναι ακριβό, αλλά παράλληλο. Στο decode, κάθε νέο token περνά μόνο του, χρησιμοποιώντας όσα έχουν υπολογιστεί ήδη. Οι δύο φάσεις έχουν πολύ διαφορετικό κόστος, και γι' αυτό οι μετρήσεις τις δίνουν χωριστά.

Η «γνώση» του μοντέλου βρίσκεται στα βάρη. Δεν ψάχνει πουθενά. Γι' αυτό ένα μικρό μοντέλο ξέρει λίγα και μπορεί να εφεύρει γεγονότα με απόλυτη σιγουριά.

Ο tokenizer: byte-level BPE

Το μοντέλο δεν βλέπει γράμματα αλλά αριθμούς από ένα λεξιλόγιο 49.152 tokens. Ο tokenizer κόβει το κείμενο σε κομμάτια με τον αλγόριθμο Byte-Pair Encoding.

1. Τα ειδικά tokens, όπως το <|im_start|>, αναγνωρίζονται πρώτα.
2. Ένα regex τύπου GPT-2 χωρίζει λέξεις, αριθμούς και σημεία στίξης. Τα ψηφία χωρίζονται ένα-ένα.
3. Κάθε κομμάτι γίνεται bytes UTF-8.
4. Τα πιο συχνά ζεύγη συγχωνεύονται διαδοχικά, με τη σειρά του αρχείου merges.

```
// Διδακτικό: μία συγχώνευση BPE.
while (true) {
  const best = lowestRankPair(parts, ranks);
  if (!best) break;
  parts = mergeAll(parts, best);
}
```

Χωρίς \p{L}

Το regex του GPT-2 χρησιμοποιεί κατηγορίες Unicode, όπως «οποιοδήποτε γράμμα». Οι παλιοί browsers δεν τις υποστηρίζουν. Το repository παράγει πίνακες Unicode από πριν και κάνει τον έλεγχο με κώδικα ES5.

Ελληνικά και κόστος

Το λεξιλόγιο φτιάχτηκε κυρίως από αγγλικά. Μια ελληνική λέξη σπάει σε πολλά μικρά κομμάτια, μερικές φορές σε μεμονωμένα bytes. Αυτό εξηγεί εν μέρει γιατί το μοντέλο τα πάει άσχημα στα ελληνικά.

Ο tokenizer ελέγχθηκε με 40.062 κείμενα, από τα οποία 40.000 τυχαία. Όλα έδωσαν ίδια αποτελέσματα με το Transformers.js.

Το chat template

Ένα μοντέλο οδηγιών έχει μάθει να διαβάζει τη συζήτηση σε συγκεκριμένη μορφή. Κάθε μήνυμα κλείνεται σε ειδικά tokens που δηλώνουν ποιος μιλά.

```
<|im_start|>system
You are Zagreus, a friendly assistant.
Answer briefly.<|im_end|>
<|im_start|>user
Who wrote the Odyssey?<|im_end|>
<|im_start|>assistant
```

Το prompt τελειώνει ανοίγοντας τη σειρά του assistant. Το μοντέλο συνεχίζει από εκεί και σταματά όταν γράψει `<|im_end|>`.

Ρόλοι

Ρόλος	Ποιος γράφει	Σκοπός
system	η εφαρμογή	ταυτότητα, ύφος, όρια
user	εσύ	η ερώτηση
assistant	το μοντέλο	η απάντηση

Ιστορικό

Για να θυμάται τι ειπώθηκε, ο Ζαγρεύς στέλνει και τα προηγούμενα μηνύματα. Κρατά μόνο τα τελευταία πέντε, με `slice(-5)`, όπως η σελίδα αναφοράς. Σε κάθε γύρο όλο το prompt υπολογίζεται από την αρχή. Η κβάντιση των ενεργοποιήσεων γίνεται ανά τανυστή, οπότε εξαρτάται από όλο το prompt και μια παλιά KV cache δεν θα έδινε ακριβώς τα ίδια αποτελέσματα.

Μέσα στο μοντέλο

Το SmolLM2-135M είναι transformer τύπου Llama. Κάθε token γίνεται ένα διάνυσμα 576 τιμών και περνά από 30 όμοια layers.

Μέγεθος	Τιμή
Layers	30
Πλάτος (hidden)	576
Κεφαλές attention	9 για τα Q, 3 για τα K και V (GQA)
Πλάτος κεφαλής	64
MLP	1.536, με SiLU και πύλη
Λεξιλόγιο	49.152
Θέσεις	RoPE

Ένα layer

```
// Διδακτικό: ένα layer τύπου Llama.
h = x + attention(rmsNorm(x));
y = h + mlp(rmsNorm(h));
// mlp(v) = down( silu(gate(v)) * up(v) )
```

Κάθε layer προσθέτει κάτι στο ρεύμα και δεν το αντικαθιστά. Αυτό είναι το residual. Το attention μαζεύει πληροφορία από προηγούμενα tokens. Το MLP τη μετασχηματίζει σε κάθε θέση χωριστά.

GQA

Με Grouped-Query Attention, τρεις κεφαλές ερωτήσεων μοιράζονται ένα ζεύγος keys και values. Η μνήμη για την KV cache πέφτει στο ένα τρίτο, με μικρή απώλεια ποιότητας.

Κβάντιση q8 στην πράξη

Στο αρχείο q8 τα βάρη των γραμμικών layers είναι uint8, με κλίμακα και zero point. Οι ενεργοποιήσεις κβαντίζονται τη στιγμή της εκτέλεσης με τον κόμβο `DynamicQuantizeLinear`. Ο πολλαπλασιασμός γίνεται με ακέραιους (`MatMulInteger`) και μόνο στο τέλος επιστρέφει σε float.

```
// Διδακτικό DynamicQuantizeLinear.
const lo = Math.min(0, min(x)), hi = Math.max(0, max(x));
const scale = (hi - lo) / 255;
const zp = clamp(roundHalfEven(-lo / scale), 0, 255);
q[i] = clamp(roundHalfEven(x[i] / scale) + zp, 0, 255);

// MatMulInteger: ακριβές ακέραιο άθροισμα.
acc += (q[k] - zp) * (w[k][j] - wzp);
y[j] = acc * scale * wscale;
```

Γιατί «half to even»

Το 2,5 στρογγυλεύει στο 2 και το 3,5 στο 4. Είναι ο κανόνας του ONNX Runtime. Αν γραφόταν `Math.round`, μια μόνο τιμή στο ,5 θα άλλαζε, και μετά από 30 layers θα μπορούσε να αλλάξει ολόκληρη λέξη.

Ακέραιοι = ακρίβεια

Τα ακέραια αθροίσματα είναι ακριβή, όποια σειρά κι αν ακολουθηθεί. Εδώ βρίσκεται το κλειδί της ταύτισης: το μόνο που πρέπει να αντιγραφεί πιστά είναι η σειρά των λίγων πράξεων float που ακολουθούν.

RMSNorm, RoPE και SiLU

RMSNorm

Πριν από το attention και το MLP, κάθε διάνυσμα κανονικοποιείται. Διαιρείται με τη ρίζα του μέσου τετραγώνου του και πολλαπλασιάζεται με ένα μαθημένο βάρος. Στο ONNX εμφανίζεται ως `SimplifiedLayerNormalization`, ή `Skip…` όταν ενώνεται με την πρόσθεση του residual.

```
// Διδακτικό RMSNorm.
let ss = 0;
for (const v of x) ss += v * v;
const r = 1 / Math.sqrt(ss / x.length + eps);
y = x.map((v, i) => v * r * g[i]);
```

RoPE

Το attention από μόνο του δεν ξέρει τη σειρά των λέξεων. Το RoPE περιστρέφει τα ζεύγη τιμών των Q και K κατά μια γωνία που εξαρτάται από τη θέση. Όταν συγκρίνονται δύο θέσεις, μένει μόνο η διαφορά των γωνιών τους, δηλαδή η απόσταση. Οι πίνακες cos και sin διαβάζονται από το ίδιο το αρχείο του μοντέλου.

SiLU

Το MLP χρησιμοποιεί SiLU, `x · σ(x)`. Το ONNX Runtime δεν καλεί το `exp` της βιβλιοθήκης. Χρησιμοποιεί μια δική του ρητή προσέγγιση της σιγμοειδούς. Για να βγαίνουν τα ίδια bits, η JavaScript αντιγράφει ακριβώς αυτή την προσέγγιση.

> Η ταύτιση bit προς bit δεν είναι ζήτημα μαθηματικών αλλά λεπτομέρειας. Ο ίδιος τύπος, γραμμένος αλλιώς, δίνει άλλα bits.

Attention και KV cache

Σε κάθε layer, κάθε token φτιάχνει τρία διανύσματα: query (τι ψάχνω), key (τι προσφέρω) και value (τι μεταφέρω). Το query κάθε θέσης συγκρίνεται με τα keys όλων των προηγούμενων θέσεων. Οι βαθμοί γίνονται βάρη με softmax, και τα βάρη αναμειγνύουν τα values.

```
// Διδακτικό attention, μία κεφαλή.
for (let j = 0; j <= t; j++)
  s[j] = dot(q[t], k[j]) / Math.sqrt(64);
const p = softmax(s);
out = sum over j of p[j] * v[j];
```

Η KV cache

Τα keys και τα values των προηγούμενων tokens δεν αλλάζουν. Στο decode υπολογίζονται μία φορά και φυλάσσονται. Κάθε νέο token φτιάχνει μόνο τα δικά του και διαβάζει τα υπόλοιπα από την cache. Στο SmolLM2 αυτό κοστίζει περίπου 46 KB ανά token.

Το softmax του MLAS

Το ONNX Runtime υπολογίζει το exp του softmax με δικό του πολυώνυμο. Η JavaScript το αντιγράφει. Τις μελλοντικές θέσεις, που στο αιτιατό attention παίρνουν exp ακριβώς 0, απλώς τις παραλείπει, κρατώντας ίδια τη δομή του αθροίσματος.

Στο prefill τα αθροίσματα γίνονται σε blocks των 256, όπως στο SGEMM του MLAS. Η σειρά άθροισης είναι μέρος της ταύτισης.

Bit προς bit με το ONNX Runtime

Ο στόχος του repository ήταν φιλόδοξος: όχι «σχεδόν ίδιες» απαντήσεις, αλλά ίδια logits σε κάθε βήμα. Τα αποτελέσματα των ελέγχων:

Έλεγχος	Αποτέλεσμα
Ενδιάμεσοι τανυστές ανά layer, prefill 32 tokens και 1 βήμα decode	176/176 ίδιοι
19 απαντήσεις (14 σενάρια) έναντι Transformers.js 3.8.1	19/19 ίδιες με 1, 3 και 4 νήματα
Οι ίδιες απαντήσεις στη σελίδα αναφοράς σε Chromium	19/19 ίδιες
Tokenizer, 40.062 κείμενα	όλα ίδια

Γιατί δεν εξαρτάται από τη μηχανή

Η πρόσθεση, ο πολλαπλασιασμός, η διαίρεση, η `sqrt`, η μετατροπή σε float32 και οι ακέραιες πράξεις ορίζονται πλήρως από το IEEE-754 και το ECMAScript. Δίνουν τα ίδια bits σε V8, SpiderMonkey και JavaScriptCore. Το `Math.exp` δεν ορίζεται ακριβώς, και γι' αυτό δεν χρησιμοποιείται εκεί που μετρά.

Σενάρια ελέγχου

Τα σενάρια περιλαμβάνουν μαθηματικά, ελληνικά, emoji, ιαπωνικά, κώδικα, πολλαπλά κενά, ειδικά tokens μέσα στο κείμενο του χρήστη, prompt 245 tokens και συζητήσεις τριών και τεσσάρων γύρων.

Η σελίδα `verify.html` τρέχει τους ίδιους ελέγχους στη συσκευή σου και συγκρίνει token προς token.

Greedy με repetition penalty

Η παραγωγή είναι ντετερμινιστική. Δεν υπάρχει τυχαία δειγματοληψία: σε κάθε βήμα επιλέγεται το token με τη μεγαλύτερη βαθμολογία. Αν δύο είναι ίσα, κερδίζει το πρώτο. Η ίδια ερώτηση με το ίδιο ιστορικό δίνει πάντα την ίδια απάντηση.

Η ποινή επανάληψης

Τα μικρά μοντέλα πέφτουν εύκολα σε βρόχους: «I think that I think that…». Η ποινή επανάληψης μικραίνει τη βαθμολογία κάθε token που υπάρχει ήδη στην ακολουθία, είτε στο prompt είτε στην απάντηση.

```
// Repetition penalty 1,12.
for (const id of new Set(allIds)) {
  const v = logits[id];
  logits[id] = v > 0 ? v / 1.12 : v * 1.12;
}
const next = firstArgmax(logits);
```

Οι θετικές βαθμολογίες διαιρούνται και οι αρνητικές πολλαπλασιάζονται. Έτσι και οι δύο απομακρύνονται από την κορυφή.

Πότε σταματά

Η παραγωγή τελειώνει στο <|im_end|> ή όταν φτάσει το όριο tokens: 96 στη σελίδα αναφοράς και στη γραπτή συζήτηση του Ζαγρέα, 60 μέσα σε κλήση. Το τελικό κείμενο περνά από `trim()`.

Ένα γρήγορο και ακριβές lm_head

Το τελευταίο βήμα κάθε token, το lm_head, πολλαπλασιάζει το διάνυσμα των 576 τιμών με τον πίνακα embedding: 49.152 γραμμές. Είναι η ακριβότερη πράξη του decode, γιατί δεν μοιράζεται με τίποτα.

Η ιδέα

Για να επιλεγεί το επόμενο token, χρειάζεται μόνο το μέγιστο, όχι όλα τα logits με ακρίβεια. Το repository κάνει δύο περάσματα:

1. Εκτιμά κάθε logit σε double, γρήγορα, μαζί με ένα αυστηρό άνω φράγμα του σφάλματος.
2. Υπολογίζει ακριβώς, σε float32 όπως το ONNX Runtime, μόνο όσα tokens μπορεί να είναι το μέγιστο μετά την ποινή επανάληψης.

```
// Διδακτικό.
const est = roughLogits(h);          // double
const best = max(est.map(e => e.v - e.err));
const cand = ids.filter(i => est[i].v + est[i].err >= best);
const next = exactArgmax(h, cand);   // float32
```

Το επιλεγμένο token είναι πάντα αυτό που θα διάλεγε η πλήρης ακριβής σύγκριση. Η βελτιστοποίηση δεν αλλάζει το αποτέλεσμα, μόνο τον χρόνο.

> Μια βελτιστοποίηση που αλλάζει την έξοδο είναι άλλο μοντέλο. Μια βελτιστοποίηση με απόδειξη ότι δεν την αλλάζει είναι απλώς ταχύτερος κώδικας.

Νήματα: workers και MessageChannel

Ένας Web Worker τρέχει σε δικό του νήμα. Το repository μοιράζει το μοντέλο σε έναν worker συντονισμού και σε αρκετούς workers υπολογισμού. Επικοινωνούν απευθείας μεταξύ τους μέσω MessageChannel.

Worker	Ρόλος
smollm-worker.js	Φόρτωση, tokenizer, παραγωγή, residual, norms
smollm-compute.js	Ένα κομμάτι από κάθε πίνακα βαρών

Κάθε compute worker κρατά ένα κομμάτι των εξόδων κάθε γραμμικού layer και υπολογίζει ολόκληρες τιμές εξόδου. Επειδή καμία τιμή δεν μοιράζεται ανάμεσα σε workers, η σειρά των πράξεων μένει ίδια. Το αποτέλεσμα είναι ίδιο με 1, 2, 3 ή 4 νήματα.

Νήματα	Prompt 37 tokens	Decode
1	2,6 s	5,2 tokens/s
2	1,6 s	11 tokens/s
4	1,5 s	13 tokens/s

Καθαρή JavaScript, Node 24, χωρίς τον χρόνο του lm_head.

Ο Ζαγρεύς ξεκινά τόσους compute workers όσοι είναι οι πυρήνες της CPU, όπως τους αναφέρει ο browser.

Το SmolLM2 σε WebAssembly

Ο πολλαπλασιασμός q8 είναι το μεγαλύτερο κόστος. Ο Ζαγρεύς αντικαθιστά τη μέθοδο `_mm` κάθε κομματιού με τον πυρήνα `smol_matmul` σε Rust και WebAssembly SIMD. Τα βάρη αντιγράφονται μία φορά στη μνήμη του WASM, τη στιγμή που φορτώνονται.

```
// Πώς «κουμπώνει» ο πυρήνας (διδακτικό σχήμα).
Shard.prototype.setWeight = function (name, u8) {
  this.ptr[name] = copyIntoWasm(u8);
};
Shard.prototype._mm = function (name, xq, ...) {
  wasm.smol_matmul(this.ptr[name], xqPtr, ...);
};
```

Τα ακέραια αθροίσματα γίνονται ακριβώς με SIMD των 128 bit. Η μετατροπή σε float γίνεται με την ίδια σειρά με τη JavaScript. Το αποτέλεσμα είναι **ίδιο token προς token**.

Μέτρηση	JS	WASM SIMD
Node, matmul	×1	×3,1–3,2
Browser, decode	3,4 tokens/s	21,5 tokens/s
Browser, μία απάντηση	37 s	5,3 s

Η επιτάχυνση στον browser είναι μεγαλύτερη απ' ό,τι στο Node. Οι πολλοί workers ανταγωνίζονται για τη μνήμη, και ο συμπαγής βρόχος του WASM χρειάζεται λιγότερες προσβάσεις σε αυτήν.

Πώς μιλά ο Ζαγρεύς

Το SmolLM2 είναι η τελευταία λύση του εγκεφάλου, όχι η πρώτη. Καλείται όταν η πρόθεση είναι ελεύθερη συζήτηση («chit») και κανένας φθηνότερος δρόμος δεν ταιριάζει.

- Ένα σύντομο system prompt ορίζει το όνομα και το ύφος: σύντομες, φιλικές απαντήσεις.
- Το ιστορικό κόβεται στα τελευταία πέντε μηνύματα.
- Το όριο είναι 96 νέα tokens στη γραπτή συζήτηση και 60 σε κλήση.
- Αν η απάντηση κοπεί από το όριο, η μισή τελευταία πρόταση αφαιρείται.

Ροή προς τη φωνή

Τα tokens φτάνουν ένα-ένα μέσω του `onToken`. Σε κλήση, ο Ζαγρεύς ψάχνει το τέλος κάθε πρότασης (τελεία, ερωτηματικό, θαυμαστικό) και στέλνει την πρόταση στο meSpeak αμέσως. Ο χρήστης ακούει την αρχή της απάντησης ενώ το τέλος ακόμη γράφεται.

```
// Διδακτικό σχήμα της ροής.
await brain.chat(text, partial => {
  const end = lastSentenceEnd(partial);
  if (end > spoken) {
    speaker.say(partial.slice(spoken, end));
    spoken = end;
  }
});
```

Ένα κουμπί «διακοπή» ή μια νέα ερώτηση σταματά την παραγωγή στο επόμενο token.

Όρια του λόγου

Λίγη γνώση

135 εκατομμύρια παράμετροι δεν χωρούν πολλά γεγονότα. Το μοντέλο ξέρει γενικότητες και γράφει ευχάριστα, αλλά συχνά εφευρίσκει ονόματα, ημερομηνίες και αριθμούς. Γι' αυτό οι ερωτήσεις γνώσεων, τύπου «ποιος», «τι είναι», «πότε», πηγαίνουν πρώτα στη Wikipedia, όχι στο SmolLM2.

Αγγλικά

Η εκπαίδευση έγινε κυρίως σε αγγλικά. Στα ελληνικά οι απαντήσεις είναι συχνά ακατανόητες. Όταν η ερώτηση είναι ελληνική, ο Ζαγρεύς δεν τη στέλνει στο γλωσσικό μοντέλο. Δοκιμάζει τη Wikipedia ή δίνει μια έτοιμη απάντηση.

Μνήμη

Το μοντέλο δεν θυμάται τίποτα πέρα από τα πέντε τελευταία μηνύματα. Για ό,τι θέλεις να μείνει, ο Ζαγρεύς έχει σημειώσεις (τεύχος IV, σελίδα 60).

Υπολογισμοί

Ένα γλωσσικό μοντέλο δεν λογαριάζει, μαντεύει ψηφία. Οι αριθμητικές πράξεις δεν φτάνουν ποτέ σε αυτό. Τις λύνει ένας αναλυτής εκφράσεων με τον αλγόριθμο shunting-yard, χωρίς `eval`.

> Ο καλύτερος τρόπος να χρησιμοποιήσεις ένα μικρό μοντέλο είναι να μην του ζητάς ό,τι μπορεί να κάνει καλύτερα ένα κομμάτι απλού κώδικα.

Πρακτική δοκιμή · ο λόγος

Μια μικρή διαδικασία

1. Γράψε «Tell me a fun fact about octopuses.» και μέτρησε τον χρόνο.
2. Ρώτα ξανά ακριβώς το ίδιο. Η απάντηση πρέπει να είναι ίδια.
3. Ρώτα «And how many hearts do they have?» για να δεις το ιστορικό.
4. Κλείσε το WebAssembly στις ρυθμίσεις, ξαναφόρτωσε και επανάλαβε το βήμα 1.
5. Ρώτα «Πόσο κάνει 17 * 23;». Ποιος απαντά;

Τι αξίζει να παρατηρήσεις

Με και χωρίς WASM το κείμενο πρέπει να είναι ίδιο, γράμμα προς γράμμα. Αλλάζει μόνο η ταχύτητα. Η ερώτηση του βήματος 5 δεν φτάνει ποτέ στο γλωσσικό μοντέλο.

```
// Φύλλο παρατήρησης · συμπλήρωσέ το.
{
  "question": "",
  "answer": "",
  "tokensPerSecond": { "wasm": null, "js": null },
  "sameText": null,
  "step5AnsweredBy": ""
}
```

Στο επόμενο τεύχος συναντάμε το μοντέλο που δεν γράφει τίποτα, αλλά καταλαβαίνει: ένα μοντέλο embeddings που φέρνει κοντά προτάσεις με το ίδιο νόημα, σε πάνω από πενήντα γλώσσες.

Multilingual MiniLM · το νόημα

Το *paraphrase-multilingual-MiniLM-L12-v2* της βιβλιοθήκης sentence-transformers μετατρέπει μια πρόταση σε 384 αριθμούς. Προτάσεις με το ίδιο νόημα παίρνουν παρόμοια διανύσματα, ακόμη κι αν είναι σε διαφορετικές γλώσσες.

Ιδιότητα	Τιμή
Αρχείο	Xenova/paraphrase-multilingual-MiniLM-L12-v2, q8
Γλώσσες	50+, μαζί με τα ελληνικά
Layers	12
Πλάτος	384, 12 κεφαλές των 32
Λεξιλόγιο	250.037 γραμμές
Μέγιστο μήκος	512 tokens
Βάρη	118 MB, λήψη ~123 MB

Ο σιωπηλός πρωταγωνιστής

Ο χρήστης δεν βλέπει ποτέ αυτό το μοντέλο να «μιλά». Όμως σχεδόν κάθε μήνυμα περνά πρώτα από αυτό. Αποφασίζει τι ζήτησες, βρίσκει τη σωστή σημείωση, διαλέγει το σωστό άρθρο της Wikipedia και διαλέγει τον τομέα εργαλείων για το Needle.

Είναι επίσης ο λόγος που ο Ζαγρεύς καταλαβαίνει ελληνικά, παρότι το γλωσσικό μοντέλο και η ακοή του δεν τα ξέρουν.

Το repository *paraphrase-multilingual-MiniLM-L12-v2_PureJS* δίνει **bit προς bit** τα ίδια embeddings με το Transformers.js.

Τι είναι ένα embedding πρότασης

Φαντάσου έναν χάρτη όπου κάθε πρόταση είναι ένα σημείο. Οι προτάσεις που λένε το ίδιο πράγμα βρίσκονται κοντά, ακόμη κι αν δεν έχουν καμία κοινή λέξη. Ο χάρτης έχει 384 διαστάσεις αντί για δύο, αλλά η ιδέα είναι η ίδια.

Πρόταση A	Πρόταση B	Σχέση
Τι ώρα είναι;	What time is it?	πολύ κοντά
Τι ώρα είναι;	Πες μου την ώρα	πολύ κοντά
Τι ώρα είναι;	Τι καιρό κάνει;	μακριά

Paraphrase

Το μοντέλο εκπαιδεύτηκε σε ζεύγη παραφράσεων. Αρχικά ένα αγγλικό μοντέλο-δάσκαλος έμαθε να φέρνει κοντά τις παραφράσεις. Μετά ο πολύγλωσσος μαθητής έμαθε να βάζει τη μετάφραση μιας πρότασης στο ίδιο σημείο με το αγγλικό πρωτότυπο. Έτσι ο χώρος έγινε κοινός για όλες τις γλώσσες.

Σύγκριση

Η ομοιότητα μετριέται με cosine, όπως στην όραση. Επειδή τα embeddings κανονικοποιούνται σε μήκος 1, το cosine είναι απλώς το εσωτερικό γινόμενο.

> Το embedding δεν «καταλαβαίνει» με την ανθρώπινη έννοια. Μετρά πόσο μοιάζουν δύο προτάσεις με βάση εκατομμύρια παραδείγματα. Για τις περισσότερες αποφάσεις ενός βοηθού, αυτό αρκεί.

Ο tokenizer: SentencePiece Unigram

Το MiniLM κληρονομεί τον tokenizer του XLM-RoBERTa: SentencePiece Unigram με λεξιλόγιο 250 χιλιάδων κομματιών, αρκετά μεγάλο για όλα τα αλφάβητα.

1. Ειδικά tokens πρώτα: `<s> <pad> </s> <unk> <mask>`.
2. Κανονικοποίηση: αφαίρεση χαρακτήρων ελέγχου, ενοποίηση κενών, NFKC.
3. Διαχωρισμός στα κενά, και ▁ μπροστά σε κάθε λέξη.
4. Viterbi: ο συνδυασμός κομματιών με τη μεγαλύτερη συνολική πιθανότητα.
5. `<s>` … `</s>` γύρω-γύρω και κόψιμο στα 512.

```
// Διδακτικό Viterbi πάνω σε μια λέξη.
best[0] = 0;
for (let end = 1; end <= n; end++)
  for (let start = 0; start < end; start++) {
    const p = vocab.score(word.slice(start, end));
    if (p !== undefined && best[start] + p > best[end]) {
      best[end] = best[start] + p; from[end] = start;
    }
  }
```

Δικό του NFKC

Το NFKC ενοποιεί ισοδύναμες γραφές: «fi» γίνεται «fi», και ένα «ά» γραμμένο ως α με χωριστό τόνο γίνεται ένας χαρακτήρας. Οι παλιοί browsers δεν έχουν `normalize`, και οι νέοι διαφέρουν στην έκδοση Unicode. Το repository έχει δική του υλοποίηση με δεδομένα Unicode 17, ελεγμένη σε 1.793.588 περιπτώσεις.

Μέσα στο μοντέλο

Το MiniLM είναι encoder τύπου BERT: κάθε token βλέπει όλα τα υπόλοιπα, μπροστά και πίσω. Δεν γράφει κείμενο. Μόνο εμπλουτίζει την αναπαράσταση κάθε token με πληροφορία από την πρόταση.

Στάδιο	Τι γίνεται
Embeddings	λέξη + τύπος token + θέση, LayerNorm
12 layers	attention 12 κεφαλών, FFN με GELU
Pooling	μέσος όρος όλων των tokens
Κανονικοποίηση	μήκος 1

Η μνήμη του λεξιλογίου

Τα 118 MB δεν είναι κυρίως layers. Ο πίνακας λέξεων (250.037 × 384) πιάνει 96 MB, ενώ όλα τα γραμμικά layers μαζί πιάνουν μόλις 22 MB. Ο πίνακας λέξεων μένει στον worker συντονισμού, γιατί χρειάζεται μόνο μία ανάγνωση γραμμής ανά token. Τα layers μοιράζονται στους workers υπολογισμού.

```
// Διδακτικό: embedding ενός token (uint8).
const row = wordEmb.subarray(id * 384, id * 384 + 384);
for (let d = 0; d < 384; d++)
  x[d] = (row[d] - zp) * scale
       + typeEmb[d] + posEmb[pos * 384 + d];
layerNorm(x);
```

Η αποκβάντιση (`DequantizeLinear`) γίνεται μόνο για τις γραμμές που χρησιμοποιούνται, όχι για όλο τον πίνακα.

Ακέραιοι πολλαπλασιασμοί

Στα γραμμικά layers του MiniLM, τα βάρη είναι int8 με κλίμακα και zero point ανά στήλη. Οι ενεργοποιήσεις κβαντίζονται δυναμικά σε uint8. Ο πολλαπλασιασμός γίνεται με ακέραιους και μετατρέπεται σε float μία φορά στο τέλος.

```
// Διδακτικό MatMulIntegerToFloat.
let acc = 0;                        // ακέραιο, ακριβές
for (let k = 0; k < K; k++)
  acc += (a[k] - aZp) * (w[k][j] - wZp[j]);
y[j] = Math.fround(acc) * Math.fround(aScale * wScale[j])
     + bias[j];
```

Δύο ισοδύναμοι πυρήνες

Το άθροισμα μπορεί να γίνει με int32 ή με double. Και τα δύο δίνουν ακριβώς τον ίδιο ακέραιο, γιατί ένα double αναπαριστά ακριβώς κάθε ακέραιο ως 2^{53}. Στη φόρτωση, κάθε worker χρονομετρεί και τους δύο για περίπου 0,2 s και κρατά τον ταχύτερο για τη συγκεκριμένη CPU.

Ο ρόλος του Math.fround

Η JavaScript υπολογίζει σε double. Το ONNX Runtime υπολογίζει σε float32. Το `Math.fround` στρογγυλεύει στο πλησιέστερο float32 ακριβώς στα σημεία όπου θα στρογγύλευε το ONNX Runtime. Το repository έχει και εκδοχή που κάνει την ίδια στρογγυλοποίηση χωρίς `Math.fround`, για browsers που δεν το έχουν.

Attention, softmax και GELU

Το attention του MiniLM έχει 12 κεφαλές των 32 τιμών. Το ONNX Runtime ενώνει τον πολλαπλασιασμό $Q \cdot K^T$ με την κλιμάκωση σε έναν κόμβο, `FusedMatMul`, με alpha = 1/√32. Πολλαπλασιάζει δηλαδή αντί να διαιρεί. Η JavaScript κάνει το ίδιο, γιατί πολλαπλασιασμός με 1/√32 και διαίρεση με √32 δεν δίνουν πάντα τα ίδια bits.

Μάσκα

Όταν πολλές προτάσεις υπολογίζονται μαζί (batch), οι μικρότερες συμπληρώνονται με `<pad>`. Η μάσκα δίνει σε αυτές τις θέσεις πολύ αρνητική τιμή πριν το softmax, ώστε να μην επηρεάζουν το αποτέλεσμα.

GELU με το πολυώνυμο του MLAS

```
// BiasGelu όπως στο ONNX Runtime.
const t = (x + bias) * Math.SQRT1_2;   // όχι x / √2
y = 0.5 * (x + bias) * (1 + erfMlas(t));
```

Η `erf` υπολογίζεται με το ίδιο πολυώνυμο που χρησιμοποιεί ο πυρήνας MLAS. Το ίδιο ισχύει για το exp του softmax. Εδώ βρίσκεται όλη η δυσκολία της ταύτισης: κάθε μικρή συνάρτηση πρέπει να αντιγραφεί όπως ακριβώς είναι γραμμένη στον πηγαίο κώδικα του ONNX Runtime, όχι όπως ορίζεται στα μαθηματικά.

Ο κώδικας του ONNX Runtime που αναπαράγεται (commit 89f8206ba4) υπάρχει στο repository, στον φάκελο `tools/ortsrc`.

Mean pooling και κανονικοποίηση

Μετά τα 12 layers υπάρχει ένα διάνυσμα για κάθε token. Η πρόταση χρειάζεται ένα. Ο απλούστερος τρόπος είναι και αυτός που χρησιμοποιεί το μοντέλο: ο μέσος όρος όλων των πραγματικών tokens, χωρίς τα tokens συμπλήρωσης.

```
// Mean pooling όπως στο Transformers.js.
for (let d = 0; d < 384; d++) {
  let s = 0;                          // double
  for (let t = 0; t < T; t++)
    if (mask[t]) s += H[t * 384 + d];
  e[d] = Math.fround(s / count);      // float32
}

// Κανονικοποίηση L2.
let ss = 0;
for (let d = 0; d < 384; d++)
  ss = Math.fround(ss + Math.fround(e[d] * e[d]));
const n = Math.fround(ss ** 0.5);
for (let d = 0; d < 384; d++) e[d] = Math.fround(e[d] / n);
```

Γιατί ο μέσος όρος δουλεύει

Μετά το attention, κάθε token έχει ήδη «διαβάσει» όλη την πρόταση. Ο μέσος όρος συγκεντρώνει τις απόψεις τους. Το μοντέλο εκπαιδεύτηκε ακριβώς με αυτό το pooling, οπότε τα διανύσματά του είναι φτιαγμένα για να συγκρίνονται έτσι.

Η σειρά των πράξεων φαίνεται σχολαστική: άθροισμα σε double, αποθήκευση σε float32, άθροισμα τετραγώνων σε float32, `** 0.5`. Είναι η σειρά του Transformers.js.

Επαλήθευση bit προς bit

Τα embeddings ελέγχθηκαν απέναντι στο Transformers.js 3.8.1 και το onnxruntime-web, με `pooling: 'mean'` και `normalize: true`.

Έλεγχος	Αποτέλεσμα
40 κείμενα σε 12 γλώσσες και 3 batches με padding: ids, last hidden state, pooling, embedding	43/43 ίδια, με 1–4 νήματα, με int32 και double
Η ίδια σύγκριση σε Chromium	43/43 ίδια
Tokenizer, κείμενα ελέγχου + 40.000 τυχαία	40.094/40.094 ίδια
NFKC έναντι `String.normalize`	1.793.588/1.793.588 ίδια

Δύσκολες περιπτώσεις

Τα κείμενα ελέγχου περιλαμβάνουν κενά κάθε είδους, emoji, ειδικά tokens, το `<mask>`, μοναχικά surrogates, άγνωστους χαρακτήρες, ένα κείμενο 122 tokens και ένα 1.618 tokens που κόβεται στα 512.

Ένα ενδιαφέρον εύρημα

Μία πρόταση μόνη της και η ίδια πρόταση μέσα σε batch δίνουν ελαφρώς διαφορετικά embeddings. Στο batch, η κβάντιση των ενεργοποιήσεων υπολογίζεται σε όλο τον τανυστή, μαζί με τις άλλες προτάσεις. Και οι δύο τρόποι ταυτίζονται με τον αντίστοιχο του Transformers.js.

Το MiniLM σε WebAssembly

Ο Ζαγρεύς καλεί το MiniLM σχεδόν σε κάθε μήνυμα, οπότε κάθε χιλιοστό του δευτερολέπτου μετρά. Στο `minilm-shard.js` προστέθηκε ένα άγκιστρο, το `setSumsHook`, που αφήνει τα ακέραια αθροίσματα να υπολογιστούν από τον πυρήνα `int_sums` σε WebAssembly SIMD.

```
// Διδακτικό σχήμα του άγκιστρου.
setSumsHook(function (a, w, K, N, out) {
  wasm.int_sums(aPtr, wPtr, K, N, outPtr);
});
```

Ο πυρήνας υπολογίζει μόνο τα ακέραια αθροίσματα. Η μετατροπή σε float, το bias και η κλίμακα μένουν στη JavaScript. Επειδή τα ακέραια είναι ακριβή, τα embeddings μένουν **bit προς bit ίδια**.

Μέτρηση	JS	WASM SIMD
Node, layers	×1	×4,0–4,1
Browser, ένα embedding	340 ms	104 ms

Γιατί τόσο μεγάλη διαφορά

Ένας SIMD πολλαπλασιασμός των 128 bit επεξεργάζεται 16 τιμές int8 μαζί, και οι πολλαπλασιασμοί με διεύρυνση σε int16 και int32 γίνονται με λίγες εντολές. Η JavaScript πρέπει να κάνει κάθε γινόμενο χωριστά, και η μηχανή δεν μπορεί να μαντέψει ότι οι τιμές χωρούν σε 8 bit.

Αναγνώριση πρόθεσης

Πώς καταλαβαίνει ο Ζαγρεύς ότι το «πες μου την ώρα» και το «what's the time» ζητούν το ίδιο; Δεν έχει λίστα με όλες τις πιθανές διατυπώσεις. Έχει για κάθε πρόθεση μερικές **φράσεις-άγκυρες**, ελληνικές και αγγλικές, και συγκρίνει το μήνυμα μαζί τους.

Πρόθεση	Παράδειγμα άγκυρας
greeting	γεια σου · hello there
time / date	τι ώρα είναι · what's the date
weather	τι καιρό κάνει · will it rain
wiki	ποιος ήταν ο · tell me about
recall	τι σου είπα για · what did I save
image	τι βλέπεις · look at this
cmd_display / voice / call	σκοτεινό θέμα · stop talking · call me
chit	ελεύθερη συζήτηση

Ο κανόνας απόφασης

1. Το μήνυμα γίνεται embedding.
2. Υπολογίζεται cosine με όλες τις άγκυρες. Κάθε πρόθεση παίρνει το καλύτερο score των αγκυρών της.
3. Αν το καλύτερο score είναι κάτω από 0,5, κανένα δεν θεωρείται σίγουρο.
4. Αν δύο προθέσεις απέχουν λιγότερο από 0,06, αποφασίζει ένας κανόνας λέξεων-κλειδιών.

Υπάρχουν επίσης προθέσεις όπως thanks, bye, identity, help και joke.

Γρήγορες συγκρίσεις

Οι άγκυρες είναι δεκάδες, και κάθε μήνυμα συγκρίνεται με όλες. Αν ξαναϋπολογίζονταν κάθε φορά, η πρόθεση θα αργούσε δευτερόλεπτα. Δύο τεχνικές το αποφεύγουν.

1. Cache των αγκυρών

Τα embeddings των αγκυρών υπολογίζονται μία φορά, την πρώτη φορά που τρέχει ο Ζαγρεύς, και αποθηκεύονται στο `localStorage`. Στις επόμενες επισκέψεις φορτώνονται αμέσως. Αν αλλάξει η λίστα αγκυρών, η cache ξαναφτιάχνεται.

2. cos_scores σε WebAssembly

Όλα τα embeddings των αγκυρών μπαίνουν σε έναν πίνακα μέσα στη μνήμη του WASM, μία φορά. Για κάθε μήνυμα, ο πυρήνας `cos_scores` υπολογίζει όλα τα εσωτερικά γινόμενα με μία κλήση.

```
// Διδακτικό σχήμα της σύγκρισης.
const M = dsp.scores('anchors', anchorMatrix, 384);
const s = M(queryEmbedding);   // ένα score ανά άγκυρα
let best = argmaxPerIntent(s, anchorIntent);
```

Ο πίνακας κρατιέται με κλειδί («anchors», «notes»), οπότε δεν αντιγράφεται ξανά αν δεν αλλάξει.

> Το ακριβό κομμάτι είναι το embedding του μηνύματος (περίπου 100 ms). Η σύγκριση με όλες τις άγκυρες κοστίζει κλάσματα του χιλιοστού.

Σημειώσεις και ανάκληση

Μπορείς να πεις στον Ζαγρέα «θυμήσου ότι το κλειδί είναι στο συρτάρι». Η σημείωση αποθηκεύεται στο `localStorage` μαζί με το embedding της, κωδικοποιημένο σε base64.

Αργότερα ρωτάς «πού είναι το κλειδί;». Ο Ζαγρεύς δεν ψάχνει λέξεις. Συγκρίνει το embedding της ερώτησης με όλες τις σημειώσεις και επιστρέφει την πιο κοντινή.

```
// Διδακτικό: αναζήτηση σημείωσης.
const q = await embed(question);
const s = notes.map(n => dot(q, n.vec));
const i = argmax(s);
if (s[i] > 0.55) return notes[i].text;
```

Η σημείωση νικά

Η ερώτηση «Πού είναι το κλειδί;» μοιάζει με ελεύθερη συζήτηση ή με ερώτηση γνώσεων, και ο ταξινομητής πρόθεσης μπορεί να τη στείλει εκεί. Γι' αυτό υπάρχει μια παράκαμψη: αν κάποια σημείωση ταιριάζει με score πάνω από 0,55, η πρόθεση γίνεται «recall», όποια κι αν ήταν.

Ιδιωτικότητα

Οι σημειώσεις δεν φεύγουν από τον browser. Διαγράφονται από το πλαϊνό μενού ή με την εκκαθάριση των δεδομένων του ιστότοπου.

Επειδή η σύγκριση γίνεται στο νόημα, μια σημείωση γραμμένη στα ελληνικά βρίσκεται και με αγγλική ερώτηση.

Το σωστό άρθρο της Wikipedia

Για τις ερωτήσεις γνώσεων ο Ζαγρεύς ρωτά τη Wikipedia, ελληνική ή αγγλική ανάλογα με τη γλώσσα της ερώτησης. Η αναζήτηση της Wikipedia όμως ταξινομεί με λέξεις. Για την «Ακρόπολη» μπορεί να φέρει πρώτα ένα άλλο άρθρο με την ίδια λέξη στον τίτλο.

Αναταξινόμηση τίτλων

1. Αν ο πρώτος τίτλος είναι ακριβώς το θέμα, κρατιέται χωρίς άλλο κόστος.
2. Αλλιώς, οι τρεις πρώτοι τίτλοι γίνονται embeddings και συγκρίνονται με το θέμα.
3. Στο score προστίθενται μικρά βάρη: +0,12 για τη σειρά της Wikipedia, +0,3 για ακριβή τίτλο, −0,1 για τίτλους με παρένθεση.
4. Οι σελίδες αποσαφήνισης παραλείπονται.

Γιατί μόνο τίτλοι

Μια πρώτη εκδοχή συνέκρινε και τα αποσπάσματα κειμένου. Ήταν λίγο πιο ακριβής, αλλά χρειαζόταν περίπου 4 s. Οι τίτλοι είναι σύντομοι, οπότε τα embeddings τους κοστίζουν ελάχιστα.

Η απάντηση

Η σύνοψη του άρθρου κόβεται: οι δύο πρώτες προτάσεις γίνονται η απάντηση, και οι υπόλοιπες μπαίνουν σε κάρτα με σύνδεσμο. Σε κλήση, ο Ζαγρεύς λέει μόνο τις δύο πρώτες.

Λέξεις-κλειδιά ως δίχτυ ασφαλείας

Ο ταξινομητής embeddings είναι ευέλικτος, αλλά όχι αλάθητος. Σε πολύ σύντομα ή διφορούμενα μηνύματα, δύο προθέσεις βγαίνουν σχεδόν ισόπαλες. Τότε αποφασίζουν κανόνες με λέξεις-κλειδιά: για παράδειγμα, «ώρα» ή «time» για την ώρα, και «καιρός» ή «weather» για τον καιρό.

Το πρόβλημα του \b

Στις κανονικές εκφράσεις της JavaScript, το `\b` σημαίνει «όριο λέξης». Όμως το `\b` αναγνωρίζει μόνο λατινικά γράμματα, ψηφία και κάτω παύλα. Για τη μηχανή, η λέξη «ώρα» δεν έχει καθόλου όρια, και ο κανόνας `/\bώρα\b/` αποτυγχάνει.

```
// Όριο λέξης που ξέρει Unicode.
const UB = '(?:(?<![\\p{L}\\p{N}])(?=[\\p{L}\\p{N}])'
         + '|(?<=[\\p{L}\\p{N}])(?![\\p{L}\\p{N}]))';
function W(re) {
  return new RegExp(re.source.replace(/\\b/g, UB),
    re.flags.includes('u') ? re.flags : re.flags + 'u');
}
W(/\bώρα\b/i).test('Τι ώρα είναι;');   // true
```

Η βοηθητική συνάρτηση `W()` μετατρέπει κάθε `\b` σε έλεγχο με κατηγορίες Unicode. Έτσι οι κανόνες γράφονται απλά και δουλεύουν και στα ελληνικά.

Οι τόνοι αφαιρούνται πριν από τη σύγκριση, οπότε το «ωρα» και το «ώρα» ταιριάζουν και τα δύο.

Τα όρια του νοήματος

Άρνηση

Τα embeddings προτάσεων δεν ξεχωρίζουν καλά την άρνηση. Το «άνοιξε τη φωνή» και το «μην ανοίξεις τη φωνή» βρίσκονται πολύ κοντά, γιατί μιλούν για το ίδιο θέμα. Γι' αυτό η ακριβής εντολή αποφασίζεται από το Needle 2 ή από κανόνες, όχι από το MiniLM.

Μήκος

Το attention μεγαλώνει με το τετράγωνο του μήκους. Μια πρόταση 22 tokens θέλει περίπου 0,3 s σε καθαρή JavaScript. Ένα κείμενο 512 tokens θέλει μερικά δευτερόλεπτα. Τα μηνύματα προς βοηθό είναι σύντομα, οπότε αυτό σπάνια φαίνεται.

Κατώφλια

Τα 0,5 της πρόθεσης και τα 0,55 των σημειώσεων επιλέχθηκαν με δοκιμές. Δεν είναι πιθανότητες. Ένα πιο αυστηρό κατώφλι κάνει τον Ζαγρέα πιο συχνά «αβέβαιο». Ένα πιο χαλαρό τον κάνει να μπερδεύει πιο συχνά προθέσεις.

Νέες προθέσεις

Για να μάθει ο Ζαγρεύς κάτι καινούργιο, δεν χρειάζεται εκπαίδευση. Αρκεί να προστεθούν μερικές άγκυρες στο `brain.js` και ένας χειριστής. Η cache ξαναφτιάχνεται μόνη της στην επόμενη φόρτωση.

> Το MiniLM είναι ο δρομολογητής του Ζαγρέα. Δεν απαντά ποτέ ο ίδιος, αλλά αποφασίζει ποιος θα απαντήσει.

Πρακτική δοκιμή · το νόημα

Μια μικρή διαδικασία

1. Γράψε «Θυμήσου ότι το ποδήλατο είναι στην αποθήκη».
2. Ρώτα «Where did I leave the bike?».
3. Ρώτα με τρεις διαφορετικούς τρόπους την ώρα: ελληνικά, αγγλικά και με ορθογραφικό λάθος.
4. Ρώτα «Ποιος ήταν ο Περικλής;» και δες ποιο άρθρο επιλέχθηκε.
5. Γράψε «μην αλλάξεις το θέμα» και δες τι καταλαβαίνει.

Τι αξίζει να παρατηρήσεις

Η σημείωση βρίσκεται παρότι η ερώτηση είναι σε άλλη γλώσσα και δεν έχει καμία κοινή λέξη. Το βήμα 5 δείχνει το όριο της άρνησης. Η πρώτη φόρτωση του Ζαγρέα είναι πιο αργή, γιατί υπολογίζονται τα embeddings των αγκυρών.

```
// Φύλλο παρατήρησης · συμπλήρωσέ το.
{
  "noteFound": null,
  "timeVariantsOk": 0,
  "wikiTitle": "",
  "negationResult": "",
  "embedMs": null
}
```

Στο τελευταίο τεύχος ο Ζαγρεύς περνά από τα λόγια στις πράξεις: ένα μοντέλο 13,7 MB γράφει κλήσεις εργαλείων σε JSON, μέσα σε μια γραμματική που δεν του αφήνει περιθώριο λάθους.

Needle 2 · η πράξη

Το Needle 2 της Cactus Compute είναι ένα πολύ μικρό μοντέλο φτιαγμένο για ένα πράγμα: να διαβάζει ένα αίτημα και μια λίστα εργαλείων, και να γράφει ποιο εργαλείο πρέπει να κληθεί, με ποια ορίσματα. Λέγεται και μοντέλο κλήσης συναρτήσεων (function calling).

Ιδιότητα	Τιμή
Αρχείο	needle2.cact, ~13,7 MB
Layers	27
Residual	mHC, 4 λωρίδες
Μνήμη n-gram	Engram Rings
Κβάντιση	CQ2 στις προβολές, CQ4 στο feed-forward
Επιπλέον	κεφαλή βεβαιότητας (confidence)
Tokenizer	SentencePiece

Το repository

Το *Needle2_PureJS_release* εκτελεί το μοντέλο σε καθαρή JavaScript, ακόμη και στον Firefox ενός Android 5.1.1 με 1 GB RAM. Η κλήση εργαλείου που παράγει ταυτίζεται με τον native κώδικα αναφοράς σε ARMv7, και η μνήμη μένει στα 35–45 MB.

Αυτό το τεύχος εξηγεί τι είναι μια κλήση εργαλείου, πώς είναι φτιαγμένο το Needle 2, πώς ο Ζαγρεύς το περιορίζει ώστε να γράφει μόνο έγκυρες εντολές, και πώς όλα τα μοντέλα δένουν σε μια κλήση.

Τι είναι μια κλήση εργαλείου

Ένας βοηθός που μόνο μιλά δεν αλλάζει τίποτα. Για να ανάψει ένα φως, να αλλάξει θέμα ή να κλείσει μια κλήση, πρέπει να μεταφράσει τα λόγια σε μια δομημένη εντολή που καταλαβαίνει ο κώδικας.

Η εφαρμογή περιγράφει τα εργαλεία της σε JSON: όνομα, περιγραφή, παράμετροι. Το μοντέλο απαντά με μια κλήση:

```
// Το εργαλείο, όπως το βλέπει το μοντέλο.
{ "name": "set_voice",
  "description": "Turn the spoken voice replies on or off",
  "parameters": { "type": "object",
    "properties": { "on": { "type": "boolean" } },
    "required": ["on"] } }

// Αίτημα: "Please stop talking out loud."
<tool_call>
{"name":"set_voice","arguments":{"on":false}}
</tool_call>
```

Η εφαρμογή διαβάζει το JSON και εκτελεί τη δική της συνάρτηση. Το μοντέλο δεν εκτελεί τίποτα. Μόνο προτείνει.

Τρεις δρόμοι

Το Needle 2 μπορεί να ξεκινήσει με τρεις τρόπους: με `<tool_call>`, με `<think>` για να σκεφτεί πρώτα, ή με απλό κείμενο αν δεν ταιριάζει κανένα εργαλείο. Το runtime δεν το αναγκάζει σε κανέναν από τους τρεις. Ο δρόμος επιλέγεται από τα πραγματικά logits.

Η αρχιτεκτονική του Needle 2

Το Needle 2 μοιάζει με ένα συνηθισμένο decoder-only transformer, με τρεις ιδιαιτερότητες.

27 layers με τέσσερις λωρίδες

Όπως ο encoder του Whistle, το Needle κρατά τέσσερα παράλληλα residual ρεύματα (mHC, 4 lanes). Κάθε layer διαβάζει έναν μαθημένο συνδυασμό τους και γράφει πίσω σε όλα. Η πληροφορία έχει περισσότερους δρόμους να ταξιδέψει χωρίς να «θολώσει».

Engram Rings

Ένας κυκλικός buffer κρατά τα πρόσφατα n-grams και τέσσερις αθροιστές (tap0 έως tap3). Είναι φτηνή μνήμη για τη μορφή του κειμένου: αγκύλες, εισαγωγικά, ονόματα εργαλείων.

Κεφαλή βεβαιότητας

Μια μικρή κεφαλή attention pooling κοιτά όλη την ακολουθία και δίνει έναν αριθμό: πόσο σίγουρο είναι το μοντέλο ότι χρειάζεται εργαλείο. Το repository δίνει την ακατέργαστη τιμή, χωρίς τεχνητή βαθμονόμηση.

Αρχείο	Ρόλος
engine-full.js	Ο runtime
cact-full.js	Ανάγνωση των βαρών .cact
tokenizer-full.js	SentencePiece
needle-router.js	Ο δρομολογητής του Ζαγρέα

CQ2, CQ4 και η σειρά των πράξεων

Τα βάρη του Needle είναι κβαντισμένα με codebooks: CQ2 με 2 bit ανά βάρος στις προβολές και CQ4 με 4 bit στο feed-forward. Η ιδέα είναι ίδια με του Whistle: κάθε μικρός κωδικός δείχνει μια τιμή του codebook.

```
// Διδακτικό: ανάγνωση CQ2.
for (let j = 0; j < 4; j++) {
  const code = (byte >> (2 * j)) & 3;
  sum += activation[4 * p + j] * codebook[code];
}
```

Η ολίσθηση του layer 13

Η πρώτη JavaScript εκδοχή έδινε σωστές απαντήσεις στην αρχή, αλλά στο «think» έπεφτε σε ατέρμονες βρόχους. Η αιτία ήταν μικρές διαφορές float ανάμεσα στη JavaScript και στον κώδικα NEON του ARM. Συσσωρεύονταν μέχρι το layer 13 και άλλαζαν τη greedy επιλογή.

Η διόρθωση

Ο πυρήνας CQ2 ξαναγράφτηκε ώστε να μιμείται ακριβώς τον packed kernel NEON: ίδια ζεύγη συσσωρευτών, ίδια σειρά. Ενεργοποιήθηκε για τις κρίσιμες προβολές, όπως τα `key_proj` και `value_proj` των engram και τα `out_proj` των layers 6, 8 και 11. Το hash των engram ευθυγραμμίστηκε bit προς bit με τον C κώδικα.

Μετά τη διόρθωση, τα πρώτα 12 tokens του reasoning ταυτίζονται με το native. Μένει μια μικρή διαφορά πιο μετά, ενώ η κλήση εργαλείου ταυτίζεται πλήρως.

Οκτώ εργαλεία είναι πολλά

Η πρώτη σύνδεση του Needle με τον Ζαγρέα έδινε στο μοντέλο όλα τα εργαλεία μαζί: θέμα, φωνή, γλώσσα, κλήση και άλλα. Τα αποτελέσματα ήταν απογοητευτικά. Κάθε εντολή ήθελε 30 έως 50 δευτερόλεπτα, και οι επιλογές ήταν συχνά λάθος.

Δύο λόγοι. Ένα μεγάλο prompt με οκτώ περιγραφές εργαλείων κοστίζει πολύ στο prefill. Και ένα πολύ μικρό μοντέλο διαλέγει καλά ανάμεσα σε δύο ή τρία εργαλεία, αλλά άσχημα ανάμεσα σε οκτώ.

Προφίλ ανά τομέα

Η λύση ήταν να μοιραστεί η απόφαση. Το MiniLM, που είναι γρήγορο, διαλέγει τον τομέα. Το Needle, που είναι ακριβές, διαλέγει την ακριβή κλήση μέσα στον τομέα.

Προφίλ	Εργαλεία
display	set_dark_mode(on)
voice	set_voice(on), stop_speaking(), set_language(lang: el | en)
call	start_call(), end_call()

```
// Το system prompt κάθε προφίλ.
You control the Zagreus assistant app.
Use the appropriate available tool for
the user's request.
```

Με τα προφίλ, μια εντολή θέλει 1,3–5 s, και οι σωστές απαντήσεις ανέβηκαν σε 12 από τις 16 στις δοκιμές ελέγχου.

Περιορισμένη αποκωδικοποίηση

Ένα ελεύθερο μοντέλο μπορεί να γράψει άκυρο JSON ή ένα εργαλείο που δεν υπάρχει. Σε κάθε προφίλ, ο Ζαγρεύς ξέρει από πριν όλες τις έγκυρες κλήσεις. Το voice για παράδειγμα έχει πέντε υποψήφιες: voice_on, voice_off, stop_speaking, lang_el, lang_en.

Κάθε υποψήφια γράφεται ως tokens. Σε κάθε βήμα, το μοντέλο επιτρέπεται να διαλέξει μόνο ανάμεσα στα tokens που συνεχίζουν κάποια υποψήφια που ζει ακόμη.

```
// Διδακτικό: πεπερασμένη γραμματική.
let alive = candidates.map(c => c.ids);
let pos = 0;
while (alive.length > 1) {
  const allowed = new Set(alive.map(a => a[pos]));
  const logits = engine.step(lastToken);
  const id = argmaxOver(logits, allowed);
  alive = alive.filter(a => a[pos] === id);
  lastToken = id; pos++;
}
return alive[0];      // έγκυρη κλήση, πάντα
```

Η συντόμευση

Μόλις μείνει μία μόνο υποψήφια, η υπόλοιπη κλήση είναι γνωστή. Δεν χρειάζεται να περάσει το σώμα της από το μοντέλο. Συνήθως η απόφαση παίρνεται στο όνομα του εργαλείου ή στην τιμή `true`/`false`, μετά από λίγα βήματα.

> Η γραμματική δεν αλλάζει την κρίση του μοντέλου. Μόνο κόβει τα tokens που θα οδηγούσαν εκτός της λίστας.

Η πρώτη απόφαση: κλήση, σκέψη ή κείμενο

Πριν από τη γραμματική υπάρχει μια πιο βασική ερώτηση: θέλει το μοντέλο να καλέσει εργαλείο; Ο δρομολογητής κοιτά τα logits του πρώτου token της απάντησης.

Πρώτο token	id	Τι ακολουθεί
<tool_call>	10	περιορισμένη αποκωδικοποίηση της κλήσης
<think>	6	έως 32 tokens σκέψης, μετά πάλι επιλογή
άλλο	–	απλό κείμενο: καμία εντολή

Αν το Needle επιλέξει απλό κείμενο, ο Ζαγρεύς δεν εκτελεί τίποτα και πέφτει στη λύση του MiniLM: την ετικέτα της κοντινότερης άγκυρας μέσα στον τομέα.

Τι επιστρέφει ο δρομολογητής

```
{
  "call": {"name":"set_voice","arguments":{"on":false}},
  "label": "voice_off",
  "reasoning": null,
  "confidence": 0.71,
  "plain": null,
  "ms": 1840,
  "steps": 6
}
```

Ενδεικτική τιμή. Η βεβαιότητα δίνεται ακατέργαστη.

Το Needle χρησιμοποιείται για μη ελληνικό κείμενο. Τα ελληνικά δεν περιλαμβάνονται στην εκπαίδευσή του, οπότε οι ελληνικές εντολές αποφασίζονται απευθείας από τις άγκυρες του MiniLM.

Ζεστό prompt: snapshot και restore

Το prompt κάθε προφίλ, δηλαδή το system και οι περιγραφές των εργαλείων, είναι πάντα το ίδιο. Αλλάζει μόνο το αίτημα στο τέλος. Γιατί να υπολογίζεται από την αρχή κάθε φορά;

Όταν φορτώνει, ο worker του Needle περνά κάθε prefix από το μοντέλο στο παρασκήνιο, εκτός από το τελευταίο token. Μετά φωτογραφίζει την κατάσταση της μηχανής.

```
// Τι κρατά το snapshot (διδακτικό σχήμα).
snap = {
  position, history,
  caches: { k, v, ks, vs, slotPos },
  engramRings: { data, scale, pos },
  conf: confidenceArrays
};
// Σε κάθε αίτημα:
engine.restore(snap);
engine.prefill(lastPrefixToken + request);
```

Σε κάθε αίτημα, η μηχανή επιστρέφει στη φωτογραφία με μια αντιγραφή πινάκων και υπολογίζει μόνο τα λίγα tokens του αιτήματος.

Προτεραιότητα

Τα προφίλ ζεσταίνονται ένα-ένα στο παρασκήνιο. Αν φτάσει αίτημα πριν τελειώσουν, το αίτημα περνά πρώτο, και το ζέσταμα συνεχίζει μετά.

> Το ίδιο κόλπο χρησιμοποιούν οι μεγάλοι servers με τη μορφή του prefix caching. Εδώ γίνεται μέσα σε έναν worker του browser.

Πρακτική δοκιμή · η πράξη

Μια μικρή διαδικασία

1. Περίμενε να γράψει το πλαϊνό μενού ότι το Needle είναι έτοιμο. Τα προφίλ ζεσταίνονται στο παρασκήνιο.
2. Γράψε «Switch to dark mode, please.» και μετά «I prefer light colours.».
3. Γράψε «Please stop talking out loud.» και μετά «Speak Greek from now on.».
4. Γράψε «Hang up the call.» χωρίς να είσαι σε κλήση.
5. Γράψε στα ελληνικά «άλλαξε σε σκοτεινό θέμα».

Τι αξίζει να παρατηρήσεις

Το βήμα 2 δεν περιέχει τη λέξη «dark» στη δεύτερη φράση. Η απόφαση στηρίζεται στο νόημα. Το βήμα 4 δείχνει ότι η εφαρμογή, όχι το μοντέλο, ελέγχει αν μια εντολή έχει νόημα τη δεδομένη στιγμή. Το βήμα 5 δεν φτάνει στο Needle: αποφασίζουν οι άγκυρες του MiniLM.

Η πρώτη εντολή

Αν δώσεις εντολή αμέσως μετά τη φόρτωση, πριν ζεσταθεί το προφίλ, θα αργήσει λίγο περισσότερο. Η δεύτερη εντολή του ίδιου τομέα θα είναι αισθητά ταχύτερη.

```
// Φύλλο παρατήρησης · συμπλήρωσέ το.
{
  "command": "",
  "profile": "display | voice | call",
  "label": "",
  "ms": null,
  "routedBy": "needle | anchors"
}
```

Η κλήση από άκρη σε άκρη

Η κλήση είναι το σημείο όπου όλα τα μοντέλα δουλεύουν μαζί, σε πραγματικό χρόνο. Πατάς το πράσινο κουμπί, και ο Ζαγρεύς ακούει μέχρι να το κλείσεις.

Στάδιο	Πού τρέχει	Χρόνος
Resample 48 → 16 kHz	WASM, κύριο νήμα	~0,1 ms ανά πλαίσιο
VAD ανά 20 ms	WASM, κύριο νήμα	συνεχώς
Τέλος φράσης	–	700 ms σιωπής
Whistle, greedy	Worker + WASM	2,3 s για 11,7 s ήχου
Πρόθεση (MiniLM)	Workers + WASM	~0,1 s
Απάντηση (SmolLM2)	Workers + WASM	5,8 s για όλη
Πρώτος ήχος (meSpeak)	Worker	~330 ms

Καταστάσεις

Η σφαίρα της κλήσης δείχνει πού βρίσκεται ο Ζαγρεύς: ακούει, σε ακούει (hearing), σκέφτεται, μιλά. Η μέτρηση κάτω από τη σφαίρα δείχνει τους χρόνους του τελευταίου γύρου.

Barge-in

Ενώ ο Ζαγρεύς μιλά, ο VAD συνεχίζει να ακούει. Αν η στάθμη μείνει 18 dB πάνω από τον θόρυβο για 240 ms, η ομιλία σταματά και η νέα σου φράση γίνεται η επόμενη ερώτηση. Το κουμπί σίγασης κλείνει προσωρινά το μικρόφωνο.

meSpeak.js · η φωνή

Ο Ζαγρεύς μιλά με το meSpeak.js, μια μεταφορά του eSpeak σε JavaScript. Δεν είναι νευρωνικό μοντέλο. Είναι συνθέτης κανόνων: μετατρέπει γράμματα σε φωνήματα με λεξικά και κανόνες προφοράς για κάθε γλώσσα, και τα φωνήματα σε ήχο με σύνθεση formant.

Ρύθμιση	Τιμή
Φωνές	el (ελληνικά), en/en-us (αγγλικά)
Ταχύτητα	165 λέξεις το λεπτό
Ύψος	45
Γλώσσα φωνής	αυτόματα, από το κείμενο
Έξοδος	WAV, ως ArrayBuffer

Πώς δουλεύει στη σελίδα

1. Η απάντηση χωρίζεται σε προτάσεις.
2. Κάθε πρόταση πηγαίνει σε έναν worker meSpeak, που επιστρέφει bytes WAV.
3. Το `decodeAudioData` τα κάνει ήχο και αυτός μπαίνει στην ουρά.
4. Ένας μετρητής γενιάς ακυρώνει ό,τι είναι στην ουρά όταν πατηθεί «σταμάτα» ή γίνει barge-in.

Η φωνή ακούγεται μηχανική, αλλά είναι μικρή, ακαριαία και καταλαβαίνει ελληνικά. Η πρώτη πρόταση ακούγεται περίπου 330 ms μετά το κείμενο.

Το meSpeak.js διανέμεται με άδεια GPL. Το έργο ανήκει στον Norbert Landsteiner και βασίζεται στο eSpeak του Jonathan Duddington.

WebAssembly: οι πυρήνες

Όλοι οι πυρήνες του Ζαγρέα βρίσκονται σε ένα μικρό crate της Rust (`wasm-src/`). Μεταγλωττίζεται δύο φορές: `zagreus-simd.wasm` με SIMD των 128 bit και `zagreus-base.wasm` χωρίς. Η σελίδα διαλέγει με `WebAssembly.validate`.

Πυρήνας	Χρήση	Αποτέλεσμα
qmat_matvec	Whistle	×1,6, bit-identical
smol_matmul	SmolLM2	×3,1, ίδια tokens
int_sums	MiniLM	×4, bit-identical
cos_scores	άγκυρες, σημειώσεις	όλα τα scores σε μία κλήση
rs_init / rs_process	μικρόφωνο	polyphase resampler
vad_*	κλήση	VAD, στάθμη, θόρυβος
zg_alloc / zg_mark / zg_reset	όλοι	απλός allocator

Ο κανόνας της ταύτισης

Ένας πυρήνας WASM μπαίνει μόνο αν δίνει τα ίδια αποτελέσματα με τη JavaScript. Τα ακέραια γινόμενα είναι ακριβή. Τα float αθροίζονται με την ίδια σειρά. Τα `tools/test_*_wasm.js` το ελέγχουν σε κάθε αλλαγή.

Αν ο browser δεν έχει SIMD, τα μοντέλα μένουν σε JavaScript, γιατί ο scalar κώδικας WASM θα ήταν πιο αργός. Ο resampler και ο VAD χρησιμοποιούν και την έκδοση base. Το Orasis-2 έχει δικά του αρχεία WebAssembly από το repository του (σελίδα 26). Ο διακόπτης «WebAssembly SIMD» στις ρυθμίσεις τα κλείνει όλα.

Τρέχοντας τον Ζαγρέα

Χρειάζεται μόνο Node.js στον υπολογιστή που σερβίρει τα αρχεία. Ο server δεν έχει εξαρτήσεις και δεν υπολογίζει τίποτα. Απλώς στέλνει αρχεία.

```
npm start          # http://localhost:8780/
npm run lan        # όλο το τοπικό δίκτυο
npm test           # έλεγχοι εγκεφάλου και πυρήνων
```

Από κινητό ή άλλον υπολογιστή

Το μικρόφωνο και η κάμερα δουλεύουν μόνο σε ασφαλή σελίδα: `localhost` ή HTTPS. Με το `npm run lan`, ο server δημιουργεί μια δική του τοπική αρχή πιστοποίησης στον φάκελο `.cert/` και ανοίγει δύο θύρες.

Διεύθυνση	Χρήση
https://IP:8443/	η σελίδα, με μικρόφωνο
http://IP:8780/ca.crt	το πιστοποιητικό για τη συσκευή
http://IP:8780/	απλό HTTP, χωρίς μικρόφωνο

1. Στη συσκευή, άνοιξε το `ca.crt` και εγκατάστησέ το ως CA certificate.
2. Άνοιξε τη διεύθυνση HTTPS και επίτρεψε το μικρόφωνο.

> Ο φάκελος `.cert/` περιέχει ιδιωτικά κλειδιά. Μην τον μοιραστείς ποτέ. Όταν τελειώσεις τις δοκιμές, αφαίρεσε το πιστοποιητικό από τη συσκευή.

Μικρό γλωσσάρι

Όρος	Σημασία στο βιβλίο
Token	Κομμάτι κειμένου με αριθμό στο λεξιλόγιο
Embedding	Διάνυσμα αριθμών που αναπαριστά κείμενο ή εικόνα
Cosine	Ομοιότητα δύο διανυσμάτων, από −1 έως 1
Logits	Βαθμολογίες για κάθε πιθανό επόμενο token
Attention	Ανάμειξη πληροφορίας από άλλες θέσεις
GQA	Κεφαλές που μοιράζονται keys και values
RoPE	Περιστροφή που κωδικοποιεί τη θέση
KV cache	Αποθηκευμένα keys και values προηγούμενων tokens
Κβάντιση	Αναπαράσταση με λιγότερα bits
q8 · CQ2 · CQ4	8 bit · 2 bit · 4 bit ανά βάρος
Prefill	Επεξεργασία όλου του prompt μαζί
Decode	Παραγωγή ενός token τη φορά
Greedy	Πάντα το πιο πιθανό token
Beam search	Πολλές υποψήφιες ακολουθίες μαζί
Mel	Κλίμακα συχνοτήτων κοντά στην ακοή
VAD	Ανίχνευση φωνής στον ήχο
Web Worker	Νήμα του browser εκτός διεπαφής
WASM SIMD	Εντολές για πολλές τιμές ταυτόχρονα
Bit-identical	Ίδιο αποτέλεσμα μέχρι το τελευταίο bit
Απόσταξη	Μικρό μοντέλο μαθαίνει να μιμείται μεγάλο
Barge-in	Διακοπή της ομιλίας του βοηθού με φωνή

Πηγές και άδειες

Τα πέντε repositories

Cactus Whistle · ακοή
github.com/AlexandrosGiann/Cactus_Whistle-PureJS

Orasis-2 · όραση
github.com/AlexandrosGiann/zagreus-orasis

SmolLM2-135M-Instruct · λόγος
github.com/AlexandrosGiann/SmolLM2_135MInstruct-PureJS

multilingual MiniLM · νόημα
github.com/AlexandrosGiann/paraphrase-multilingual-MiniLM-L12-v2_PureJS

Needle 2 · πράξη
github.com/AlexandrosGiann/Needle2_PureJS_release

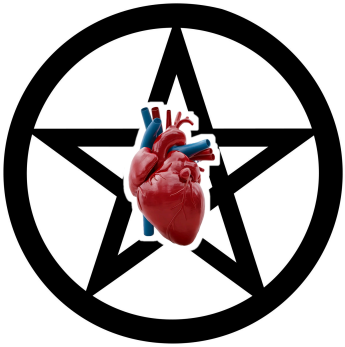
Ο Ζαγρεύς

github.com/AlexandrosGiann/Zagreus

Μοντέλα και βιβλιοθήκες

Έργο	Προέλευση
Whistle, Needle 2	Cactus Compute
Orasis-2	MobileNetV4 (timm, Apache 2.0), δάσκαλος OpenCLIP ViT-B-16 (MIT)
SmolLM2-135M-Instruct	Hugging Face
paraphrase-multilingual-MiniLM-L12-v2	sentence-transformers
meSpeak.js	N. Landsteiner, GPL
Wikipedia, Open-Meteo	δημόσια APIs, χωρίς κλειδί

Κάθε μοντέλο διέπεται από την άδεια του δημιουργού του. Λεπτομέρειες υπάρχουν στο README και στα αρχεία LICENSE κάθε repository.



ZAGREVS

Ακοή · Όραση · Λόγος · Νόημα · Πράξη

Πέντε μοντέλα σε καθαρή JavaScript,
μία καρδιά από WebAssembly.

Ζαγρεύς · Οκτώβριος 2026